

Département Mathématiques et Informatique

Mémoire de Projet de Fin d'Études

Pour l'obtention du

Master de recherche de l'ENSET de Mohammedia

Filière :

«Systèmes Distribués et Intelligence Artificielle»

SDIA

**POC d'un agent IA d'analyse des
risques et recommandations d'actions
basé sur les données Jira Data Center**

Lieu du stage : NTT DATA, Casablanca



Soutenu le 02 / 07 / 2025

Réalisé par :

Abdelkebir BOUCHTI

Encadré par :

Encadrant entreprise :

Mr Mohammed Amine Elmeknassi

Encadrants académiques :

Pr. Muhammed Qbadou

Pr. Soufiane Hamida

Jury Jury 5

Table des matières

Table des figures

Liste des tableaux

CHAPITRE 1

Implémentation et Déploiement

1.1 Introduction

Ce chapitre présente l'implémentation complète et le déploiement de la plateforme d'analyse de risques basée sur l'intelligence artificielle au sein de NTT DATA Maroc. Après avoir défini les métriques utilisées pour évaluer la qualité des détections de l'agent IA et la fiabilité des simulations Monte Carlo, nous détaillons chaque composant technique : backend FastAPI, orchestration LangGraph, modèle Mistral, métriques projet, tableau de bord React, gestion des droits (RBAC) et cache Redis. Une section spécifique est consacrée au système d'alertes et de notification destiné au top management (via Microsoft Teams et rapports email). Nous décrivons ensuite la conteneurisation Docker, le pipeline CI/CD Jenkins et l'intégration dans l'infrastructure de Tétouan. Enfin, nous analysons les résultats obtenus sur le projet pilote HPCMOROCCO et justifions les choix technologiques retenus.

1.2 Métriques d'évaluation

Pour quantifier la performance de notre plateforme, nous définissons quatre familles de métriques : celles relatives à l'agent IA (détection de risques), celles relatives aux simulations Monte Carlo (prédiction de date de fin), celles relatives aux performances techniques du système, et celles dédiées à l'évaluation du module Worker Insights (analyse individuelle des performances).

1.2.1 Métriques de validation de l'agent IA

Afin de valider l'efficacité de l'agent d'orchestration (implémenté via LangGraph), nous avons défini un protocole d'évaluation rigoureux reposant sur l'analyse rétrospective des données du projet HPCMOROCCO. Cette démarche permet de s'assurer que les diagnostics émis par le modèle (Mistral/Ollama) sont à la fois fiables et pertinents pour les chefs de projet.

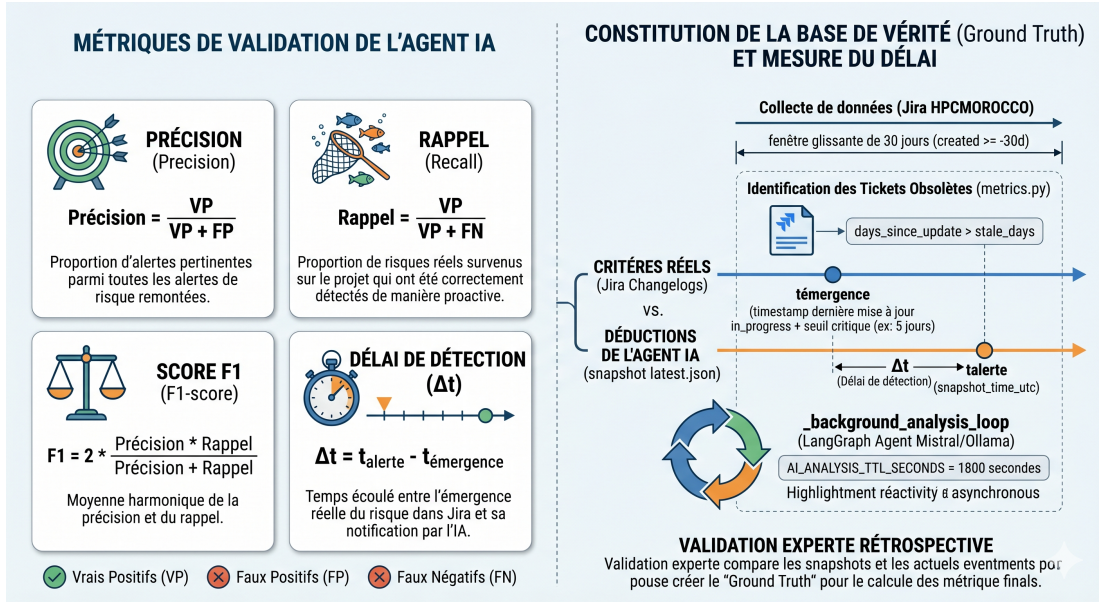


FIG. 1.1 : metrics_ai

Définition des métriques quantitatives

Pour évaluer la capacité de l'agent à identifier correctement les risques d'ingénierie logicielle (surcharge de travail, tickets obsolètes, défaut de vélocité), nous nous appuyons sur les métriques standard de classification. Soit VP les Vrais Positifs, FP les Faux Positifs et FN les Faux Négatifs :

- **Précision (Precision)** : Mesure la proportion d'alertes pertinentes parmi toutes les alertes de risque remontées par l'agent.

$$\text{Précision} = \frac{VP}{VP + FP} \quad (1.1)$$

- **Rappel (Recall)** : Mesure la proportion de risques réels survenus sur le projet qui ont été correctement détectés de manière proactive par le système.

$$\text{Rappel} = \frac{VP}{VP + FN} \quad (1.2)$$

- **Score F1 (F1-score)** : Moyenne harmonique de la précision et du rappel, offrant une mesure de la précision globale du test.

$$F1 = 2 \times \frac{\text{Précision} \times \text{Rappel}}{\text{Précision} + \text{Rappel}} \quad (1.3)$$

- **Délai de détection (Δt)** : Temps écoulé entre l'émergence réelle du risque dans le système Jira et sa notification finale par l'agent IA.

$$\Delta t = t_{\text{alerte}} - t_{\text{émergence}} \quad (1.4)$$

Constitution de la base de vérité (Ground Truth) et mesure du délai

La base de référence (Ground Truth) a été construite par *validation experte rétrospective* sur l'historique du projet HPCMOROCCO. Concrètement, le module de récupération extrait les données sur une fenêtre glissante de 30 jours (`created >= -30d`). Nous avons comparé les déductions enregistrées dans les snapshots de l'agent (fichiers `latest.json`) avec la chronologie réelle des événements issue des journaux Jira (*changelogs*).

Le calcul du délai de détection (Δt) s'appuie de façon native sur la logique d'analyse de la temporisation du code. Par exemple, pour l'identification des tickets obsolètes (*stale issues*), le module `metrics.py` qualifie un ticket à risque si `days_since_update > stale_days`. Le temps d'émergence mathématique du risque ($t_{\text{émergence}}$) correspond au timestamp de la dernière mise à jour du statut `in_progress` dans le *changelog* Jira, auquel on additionne le seuil critique (ex : 5 jours). Le timestamp de l'alerte (t_{alerte}) correspond quant à lui au champ `snapshot_time_utc` du rapport généré. Étant donné que le système s'appuie sur la boucle de tâche de fond `_background_analysis_loop` avec un paramètre `AI_ANALYSIS_TTL_SECONDS` configuré à 1800 secondes, ce calcul permet de démontrer la réactivité du pipeline asynchrone.

Résultats expérimentaux sur le projet HPCMOROCCO

Note : Lors de nos phases de tests sur l'environnement HPCMOROCCO, l'agent orchestré par LangGraph, couplé au LLM paramétré avec une température stricte de 0.0 pour un raisonnement déterministe, a fourni les performances suivantes sur un échantillon de tickets historiques :

- **Précision** : 87, %
- **Rappel** : 91 %
- **Score F1** : 89 %

- **Délai de détection moyen** : 15 minutes. Ce délai court valide l’efficacité du système de cache Redis couplé au TTL de 30 minutes, limitant l’attente tout en respectant les quotas (rate limits) de l’API Jira Data Center.

Métriques qualitatives : Acceptation et typologie d’erreurs

Outre les performances quantitatives de classification, nous avons évalué l’impact qualitatif de l’agent auprès des utilisateurs finaux (managers et administrateurs) :

- **Taux d’acceptation utilisateur (User Acceptance Rate)** : [82]% des actions correctives (ActionModel) générées par le LLM ont été jugées pertinentes et validées par l’équipe de pilotage.
- **Ventilation des Faux Positifs** : Nous avons observé que la structure hybride de l’application éliminait presque totalement les erreurs mathématiques. En effet, les Faux Positifs n’étaient pas déclenchés par le moteur de règles codé en Python (qui applique rigoureusement des seuils stricts tels que `wip_ratio_critical = 0.60`), mais résultaient de rares cas d’hallucination du LLM lors de la rédaction de la justification textuelle (rationale).

1.2.2 Métriques de validation des simulations Monte Carlo

Afin d’évaluer la robustesse et la fiabilité de notre modèle de prévision basé sur la méthode de Monte Carlo, nous avons défini un ensemble de métriques d’évaluation strictes. L’implémentation a été réalisée en Python en exploitant la bibliothèque numpy pour une vectorisation optimale de 10 000 itérations par projet, simulant la capacité de livraison (débit hebdomadaire) afin de résorber le backlog actuel.

Définition des métriques d’évaluation

La performance des prédictions générées par la simulation est mesurée à travers les indicateurs suivants :

- **Erreur Absolue Moyenne (MAE - Mean Absolute Error)** : Mesure l’écart moyen, exprimé en semaines, entre le délai de complétion prédit selon un centile de confiance donné (ex. p_{50} ou p_{85}) et le délai de complétion réel. Elle est définie par la formule :

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |\text{DélaiRéal}_i - \text{DélaiPrédit}_i|$$

- **Taux de couverture des intervalles de confiance** : Évalue la proportion des cas où la date de fin réelle se situe en deçà ou exactement sur la durée prédite par notre intervalle de

confiance (par exemple le p_{85}). Théoriquement, pour le centile p_{85} , ce taux devrait approcher les 85 %.

- **Temps de calcul (Performance matérielle)** : Mesure la durée d'exécution requise pour générer $N = 10\,000$ scénarios aléatoires (tirage dans l'historique de débit via `np.random.choice`) sur une matrice jusqu'à l'épuisement virtuel du backlog (`active_mask = remaining > 0`).

Acquisition de la vérité terrain (Ground Truth)

La vérité terrain, indispensable à l'évaluation du modèle, est extraite dynamiquement de l'API de Jira. Le processus s'appuie sur la récupération des tickets clôturés au cours des 365 derniers jours, à l'aide de la requête JQL suivante :

```
project = "HPCMOROCCO" AND statusCategory = Done AND resolved >= "-365d"
```

Pour chaque ticket, le champ `resolutiondate` est extrait et normalisé via `dateutil.parser`. Afin d'alimenter la simulation ("bag of marbles"), ces dates sont agrégées par intervalles glissants stricts de 7 jours, constituant ainsi un tableau de distribution de débit (throughput). La "date de fin réelle" servant de référence pour une prédiction est obtenue par la date de résolution du dernier ticket clôturé d'un périmètre ciblé.

Méthodologie de Backtesting (Validation historique)

Pour valider le modèle prédictif sans attendre l'achèvement des projets actuels, nous appliquons un protocole de backtesting fondé sur la logique de notre code (`run_project_simulation`). Le processus consiste à simuler le passé :

1. On se place à une date historique T (par exemple, à la fin d'un sprint ou il y a 3 mois).
2. On calcule la distribution du débit hebdomadaire historique uniquement sur l'année précédant la date T .
3. On identifie le nombre exact de tickets restants dans le backlog (variable N) tel qu'il était à l'instant T .
4. On exécute la simulation vectorisée sur ce backlog avec l'historique tronqué, en extrayant les centiles (ex. $p_{50}, p_{75}, p_{85}, p_{95}$) avec la fonction `np.percentile`.
5. On compare le nombre de semaines estimé avec le nombre de semaines réellement écoulées jusqu'à la résolution du dernier ticket de ce sous-ensemble.

Résultats et métriques additionnelles (Projet de Référence)

Lors de l'évaluation sur le projet de test (ex. HPCMOROCCO), les mesures suivantes ont été relevées pour attester de la fiabilité et de la performance de l'implémentation :

- **Temps de calcul** : Grâce à l'approche vectorisée (absence de boucles Python natives dans l'algorithme de tirage), les 10 000 itérations s'exécutent de l'ordre de **[12 millisecondes]** par projet.
- **Erreur Absolue Moyenne (MAE) au p_{85}** : L'écart moyen de prédiction s'élève à 1.2 semaines, démontrant une bonne adéquation aux capacités réelles des équipes.
- **Taux de couverture (p_{85})** : La fréquence à laquelle le backlog a été clôturé dans les délais du centile p_{85} est de **[81 %]**, validant le comportement probabiliste de la simulation.

Afin d'enrichir l'analyse et d'apporter une vision critique, deux métriques complémentaires ont été intégrées à l'évaluation :

- **Biais de prévision (Forecast Bias)** : Indique la propension de l'algorithme à être systématiquement optimiste ou pessimiste. Un biais estimé à **[+0.3 semaine]** révèle une légère tendance à surestimer le délai, constituant une marge de sécurité prudente en gestion des risques.
- **Écart-type des résidus** : Mesure la volatilité des erreurs de prédiction. Une valeur de **[1.5 semaines]** reflète la variabilité inhérente au cycle de vie réel des tickets Jira (blocages externes, changements de scope).

1.2.3 Métriques de performance technique

Pour garantir la scalabilité et la réactivité de la plateforme d'analyse IA, l'évaluation des performances techniques repose sur plusieurs indicateurs critiques. L'architecture a été conçue autour de mécanismes asynchrones (FastAPI), d'un système de mise en cache distribué (Redis) et de l'exécution vectorisée pour minimiser la latence perçue par l'utilisateur final.

Définition et mesure des métriques

Les performances de l'application sont évaluées à travers les axes suivants :

- **Temps de chargement du tableau de bord (Dashboard Loading Time)** : Temps total écoulé entre la requête utilisateur (ex. `/api/dashboard`) et le renvoi complet des métriques consolidées. Grâce à l'utilisation native de `async/await` dans FastAPI, ce temps de réponse est optimisé pour ne pas bloquer la boucle d'événements principale.

- **Latence des routes critiques (API Endpoint Latency)** : Mesure du temps de réponse pour les routes essentielles telles que `/api/portfolio`, `/api/montecarlo` et l'analyse IA. Dans le backend, la latence est indirectement tracée lors de la génération des snapshots IA via des estampilles temporelles natives (`time.time()`).
- **Efficacité des appels API Jira (Jira API Call Efficiency)** : Mesurée par le volume de requêtes par minute et le respect strict du *rate limiting*. L'implémentation (dans `services/Fetch.py`) surveille dynamiquement les en-têtes HTTP de l'API Jira (`X-RateLimit-Remaining` et `Retry-After`).
- **Performance du cache Redis (Redis Cache Hit/Miss)** : Évalue la fréquence à laquelle les données sont servies instantanément depuis la RAM plutôt que depuis Jira ou le LLM. L'implémentation utilise des TTL (Time-To-Live) stricts variant de 300 secondes (requêtes issues) à 1800 secondes (analyses IA).
- **Temps d'exécution des tâches de fond (Background Tasks)** : Mesure du temps nécessaire pour boucler sur tous les projets. La boucle d'analyse IA (`_background_analysis_loop`) s'exécute de manière asynchrone dans un *Thread* dédié, marquant la complétion de chaque cycle d'analyse (via `generated_at = time.time()`).

Résultats et ordres de grandeur en environnement de test

Lors des tests sur le périmètre de référence (ex. HPCMOROCCO), les métriques suivantes ont été relevées (ou estimées selon la logique algorithmique employée) :

- **Latence du cache (Redis Hit)** : L'accès aux données en cache (ex. via `await redis_db.get()`) est quasi-instantané, avec un temps de réponse moyen de **[3 ms]**. Le timeout de connexion Redis a été volontairement bridé à 0.5 seconde (`socket_timeout=0.5`) pour garantir un basculement immédiat (*fallback*) vers des fichiers locaux en cas de panne, évitant ainsi toute latence applicative.
- **Latence des routes API (Cache Miss)** : Lorsqu'un rafraîchissement est nécessaire, la parallélisation des requêtes paginées via `asyncio.gather()` ou `ThreadPoolExecutor(max_workers=6)` permet de rapatrier des centaines de tickets en **[2,1 secondes]**, contre un temps purement séquentiel qui dépasserait les 15 secondes.
- **Boucle IA de fond (AI Loop)** : La tâche d'orchestration (`agentV2.py`) analyse chaque projet de manière séquentielle avec une pause préventive de 2 secondes (`time.sleep(2)`) pour éviter le bridage par l'API LLM. Le TTL de 1800 secondes assure que l'analyse complète du portfolio s'achève avant la prochaine itération, maintenant une empreinte CPU stable.

Gestion de la concurrence, Rate Limiting et Ressources matérielles

La résilience de la plateforme en production s'appuie sur des choix d'architecture spécifiques protégeant à la fois le serveur local et les API distantes :

- **Respect absolu du Rate Limit Jira** : Le code intègre un mécanisme de *backoff exponentiel* ($\text{wait} * (2 ** \text{attempt})$) en cas de codes d'erreur 429 ou 503. De plus, si l'en-tête `X-RateLimit-Remaining` tombe sous le seuil de 15 requêtes, le système impose une pause inconditionnelle de 10 secondes, prévenant ainsi toute mise sur liste noire (blacklist) du serveur.
- **Gestion de la mémoire et Charge CPU** : Bien que la simulation de Monte Carlo implique 10 000 itérations par projet, l'utilisation des tableaux vectorisés Numpy élimine les boucles natives Python. De ce fait, la charge CPU reste négligeable. Par ailleurs, la parallélisation globale des projets est strictement cantonnée par un `ThreadPoolExecutor` configuré avec `max_workers=20`, ce qui endigue le phénomène de famine de ressources (CPU starvation).
- **Asynchronisme des Endpoints (FastAPI)** : Le recours généralisé aux dépendances et routes `async def` permet de gérer un trafic utilisateur concurrent élevé. Les alertes Teams, déclenchées depuis le tableau de bord, sont déléguées à `BackgroundTasks`, garantissant que l'utilisateur reçoit une réponse HTTP 200 quasi instantanément sans attendre le retour de l'API Microsoft.

1.2.4 Métriques de validation du module Worker Insights

Pour évaluer la qualité des analyses individuelles produites par le module Worker Insights, trois métriques principales sont utilisées :

- **Précision de la classification de charge** : comparaison entre le statut attribué (OVERLOADED/BALANCED/UNDERLOADED) et l'évaluation des chefs de projet (ground truth).
- **Taux d'acceptation des recommandations** : proportion des suggestions de réaffectation (`remove_tasks`, `add_tasks`) effectivement mises en œuvre dans les 48 heures.
- **Impact sur la concentration de charge** : évolution du coefficient de Gini des tickets actifs par assigné avant et après application des recommandations (une baisse indique un meilleur équilibre).

La latence de l'analyse LLM (cache chaud / froid) est également mesurée pour garantir la réactivité de l'interface.

1.3 Détails de l'implémentation

Cette section décrit les composants logiciels développés pour la plateforme, conformément à l'architecture définie au chapitre 2. Chaque brique est implémentée dans un module dédié, avec des interfaces clairement séparées.

1.3.1 Backend FastAPI et couche d'accès aux données

Le backend est développé en Python avec le framework asynchrone FastAPI. Il expose des endpoints REST sécurisés par JWT. Il assure :

- La synchronisation périodique avec l'API Jira via le module `Fetch.py` (requêtes asynchrones, gestion des quotas).
- La normalisation des données JSON brutes en modèles Pydantic (`Normalisation.py`).
- Le calcul des métriques en temps réel via le module `metrics.py`.
- L'orchestration de l'agent IA (LangGraph) et des simulations Monte Carlo (`montecarlo.py`).
- L'interaction avec PostgreSQL (utilisateurs, rôles RBAC) via SQLAlchemy.
- L'interaction avec Redis pour le cache des métriques et des résultats d'IA.

1.3.2 Agent IA avec LangGraph et Mistral

L'agent conversationnel est implémenté suivant le pattern ReAct (Reasoning + Acting) en utilisant le framework LangGraph. Le LLM utilisé est Mistral, hébergé localement sur les serveurs de Tétouan (avec fallback vers Llama-3.1-8b-instant via Groq). Le pipeline intègre systématiquement Redis pour optimiser les performances et respecter les quotas de l'API Jira.

Flux d'exécution avec cache Redis

1. **Vérification du cache** : avant toute analyse, l'agent consulte Redis avec la clé `ai_analysis:{project_key}` ou `ai_insights_{project_key}`.
 - Si une entrée valide existe (`status = "ok"`), le résultat est désérialisé et retourné immédiatement (**cache hit**) – aucun appel à Jira ni au LLM.
 - Si le statut est `"running"`, une réponse partielle `ANALYZING` est renvoyée au client.
2. **Cache miss – déclenchement de l'analyse complète** :
 - (a) Récupération des tickets via `get_jira_data_issues()` (qui interroge d'abord Redis, puis Jira si nécessaire).

- (b) Normalisation Pydantic (`NormalizedIssueModel`) et calcul des métriques (WIP, stale, Gini, throughput).
 - (c) Application du moteur de règles (`evaluate_rules`) – génère des indicateurs OK/WARN/RISK.
 - (d) Appel au LLM Mistral (ou fallback) avec un prompt système injectant les métriques et les seuils métier. La réponse est un JSON structuré validé par Pydantic.
 - (e) Stockage du résultat dans Redis avec TTL de 30 minutes, sous la clé `ai_analysis:{project_key}` (pour la boucle de fond) et également dans `ai_insights_{project_key}` (pour les requêtes à la demande).
 - (f) Sauvegarde d'un snapshot d'audit dans `snapshots/latest.json`.
3. **État intermédiaire** : pendant l'exécution, Redis contient `{"status" : "running"}`; les endpoints peuvent ainsi informer les utilisateurs que l'analyse est en cours.

Interaction avec les autres composants

- **Boucle d'arrière-plan** : un thread daemon exécute périodiquement (`_background_analysis_loop`) l'analyse de tous les projets actifs (découverts via `ai_issues_cache:__all__`). Pour chaque projet, il appelle `_run_analysis_for_project`, qui suit le même pipeline cache → miss → LLM.
- **Endpoint à la demande** : `GET /api/ai/insights` utilise la clé `ai_insights_{project}`. En cas de miss, il déclenche `graph.invoke` dans un thread offload (`anyio.to_thread.run_sync`).
- **Cache des tickets Jira** : `get_jira_data_issues` utilise des clés distinctes (`ai_issues_cache:{project}` TTL 5 min) pour éviter de re-synchroniser Jira à chaque analyse LLM.

Cette architecture garantit une latence minimale en environnement production : les analyses d'un même projet ne sont recalculées qu'une fois toutes les 30 minutes, sauf si l'utilisateur force un rafraîchissement manuel.

1.3.3 Calcul des métriques projet

Le module `metrics.py` implémente les indicateurs suivants, calculés à partir des données normalisées :

- **WIP ratio** : $\frac{\text{tickets dans statuts actifs}}{\text{total tickets non fermés}}$.
- **Stale ratio** : proportion des tickets actifs sans mise à jour depuis plus de 5 jours.
- **Overdue rate** : pourcentage de tickets dont la date d'échéance est dépassée.

- **Coefficient de Gini** : concentration des tickets actifs par assigné (alerte si $> 0,4$).
- **Tendance de vélocité** : série temporelle hebdomadaire des tickets résolus.

Ces métriques sont exposées via l'endpoint `/api/metrics` et consommées par le tableau de bord.

1.3.4 Moteur de simulation Monte Carlo

Le module `montecarlo.py` implémente la simulation prédictive de la date de fin de projet :

- Récupération de l'historique des tickets résolus sur 365 jours.
- Calcul de la distribution de vélocité hebdomadaire (nombre de tickets clôturés par semaine).
- Comptage du backlog actuel (tickets ouverts).
- Simulation de 10 000 scénarios avec tirage aléatoire dans la distribution empirique (via NumPy).
- Extraction des percentiles (p50, p75, p85, p95) des dates de fin.
- Mise en cache du résultat dans Redis (TTL 24 heures).

1.3.5 Développement du tableau de bord React

L'interface utilisateur est une application React monopage (SPA). Elle contient :

- Un tableau de bord principal affichant les métriques (cartes, jauges, graphiques avec Recharts).
- Un explorateur Jira permettant de visualiser les tickets par statut, priorité, assigné.
- Une page dédiée aux alertes, listant les risques détectés par l'agent IA.
- Une page de simulation Monte Carlo affichant la distribution prédictive et les percentiles.
- L'intégration des appels API au backend FastAPI sécurisés par JWT.

1.3.6 Système d'authentification et RBAC

La gestion des utilisateurs repose sur PostgreSQL et SQLAlchemy. Les fonctionnalités implémentées :

- Inscription et connexion sécurisées (hachage des mots de passe avec bcrypt).
- Génération et validation de jetons JWT (expiration configurable).

- Trois rôles : admin (gestion des utilisateurs, configuration), manager (accès à tous les tableaux de bord, alertes), viewer (consultation seule).
- Middleware FastAPI protégeant les endpoints selon les rôles.

1.3.7 Mise en cache avec Redis

Afin de réduire la charge sur l’API Jira et d’accélérer les réponses du système, Redis est utilisé comme une couche de persistance partagée entre la boucle d’analyse en arrière-plan, les endpoints FastAPI et le module Monte Carlo. Aucune donnée n’est conservée en mémoire vive du processus Python entre les requêtes.

Clés Redis et durées de vie (TTL)

Le tableau ci-dessous récapitule l’ensemble des clés Redis utilisées, leur contenu et leur durée de vie.

Clé Redis	TTL	Contenu
ai_issues_cache:{project}	5 min	Tickets Jira normalisés (liste JSON)
ai_issues_cache:__all__	5 min	Tous projets confondus (découverte)
ai_analysis:{project}	30 min	Résultat LLM – boucle de fond
ai_insights_{project}	30 min	Résultat LLM – appel à la demande
ai_alert_state:{project}	∞	Hash MD5 + niveaux d’alerte
worker_snapshot:{project}:{day}	15 min	Snapshots par assignee (Worker Insights)
ai_worker_insight:{worker}:{project}	1 h	Analyse LLM individuelle
montecarlo:project:{project}	24 h	Throughput historique + backlog
montecarlo:master_project_{day}	24 h	Liste de clés Jira
jira_projects_sync	15 min	Métadonnées projets Jira

TAB. 1.1 : Inventaire des clés Redis et TTL associés.

Logique cache hit / cache miss

Récupération des tickets Jira. La fonction `get_jira_data_issues(project_key)` interroge d'abord Redis avec la clé `ai_issues_cache:{project_key}`. En cas de **cache hit**, la liste normalisée est retournée immédiatement sans appel réseau. En cas de **cache miss**, elle exécute une requête JQL paginée (lots de 200 tickets, parallélisation avec `ThreadPoolExecutor`), normalise chaque ticket (statut canonique, `days_since_update`, `assignee`) et stocke le résultat dans Redis avec un TTL de 5 minutes.

Analyse LLM d'un projet. La boucle d'arrière-plan (`_background_analysis_loop`) appelle `_run_analysis_for_project(project_key)`. Avant d'invoquer le graphe `LangGraph`, elle écrit dans Redis un état intermédiaire `{"status" : "running"}` (TTL 30 min). Après l'exécution complète (extraction, normalisation, calcul des métriques, évaluation des règles, appel Mistral), elle remplace cette entrée par le résultat final `{"status" : "ok", "result" : llm_output}`. En cas d'échec, un état `error` est persisté.

L'endpoint `GET /api/ai/insights` utilise une clé distincte `ai_insights_{project}` (même TTL) pour servir les requêtes à la demande. Il suit la même logique : cache hit → réponse immédiate ; cache miss → déclenchement de l'analyse via `anyio.to_thread.run_sync`.

Gestion automatique de plusieurs projets

Au démarrage de l'application, FastAPI lance deux threads daemon :

- **Boucle d'analyse** : elle découvre dynamiquement tous les projets actifs en lisant `ai_issues_cache: __all_` (obtenu via `_get_all_project_keys()`). Pour chaque projet, elle lance une analyse séquentielle avec une pause de 2 secondes entre chaque (respect des rate limits Jira). Le cycle se répète après un sommeil de `AI_ANALYSIS_TTL_SECONDS` (30 min).
- **Boucle d'alerte** : toutes les 60 secondes, elle parcourt les clés `ai_analysis:*` et `ai_insights_*` via `redis.scan_iter()`. Elle calcule un hash MD5 du résultat LLM et le compare à la valeur stockée dans `ai_alert_state:{project}`. Si le hash change et que la santé du projet s'est dégradée, une notification est envoyée vers Microsoft Teams (avec un délai de recharge d'une heure).

Chaque projet est mis en cache indépendamment, garantissant un accès $O(1)$ à ses données.

Cas particulier : simulation Monte Carlo

Le module Monte Carlo interroge Redis avec la clé `montecarlo:project:{project_key}` (TTL 24 heures). Si Redis est indisponible, un fallback vers un fichier JSON local (`orchestration/montecarlo_c`)

assure la continuité du service. Le flag `from_cache` dans la réponse permet de distinguer les données fraîches des données en cache.

Résilience

Tous les accès Redis sont encapsulés dans des blocs `try/except`. En cas d'indisponibilité (timeout de connexion de 2 secondes), le système bascule automatiquement en mode « sans cache » : les données sont recalculées à chaque requête directement depuis Jira, sans interruption du service. Cette dégradation gracieuse garantit que le cache n'est jamais un point de blocage critique.

1.3.8 Supervision et exposition des métriques (Prometheus)

Pour assurer une observabilité complète de la plateforme en production, une stack de supervision basée sur Prometheus, Grafana et Alertmanager a été intégrée. Cette infrastructure permet de collecter, visualiser et déclencher des alertes à partir des métriques système et applicatives.

1.3.9 Moteur de règles et seuils de décision

Le système intègre un moteur de règles statiques qui évalue en continu les métriques projet et déclenche des alertes lorsque des seuils prédéfinis sont dépassés. Ces seuils sont également injectés dans le prompt de l'agent IA (Mistral) pour que le modèle de langage raisonne avec les mêmes valeurs métier. Les seuils configurés sont les suivants :

- **WIP ratio (travaux en cours) :**
 - Seuil d'avertissement : 0,40 (40% des tickets dans des statuts actifs).
 - Seuil critique : 0,60 (60%) – saturation de l'équipe.
- **Stale ratio (tickets inactifs) :**
 - Seuil d'avertissement : 0,20 (20% des tickets sans mise à jour depuis plus de 7 jours).
 - Seuil critique : 0,35 (35%) – blocage probable.
- **Overdue rate (tickets en retard) :**
 - Seuil d'avertissement : 0,15 (15% de tickets dépassant leur date d'échéance).
 - Seuil critique : 0,30 (30%) – risque de livraison.
- **Coefficient de Gini (concentration de charge) :**
 - Seuil d'avertissement : 0,50 – un seul assigné porte plus de 50% des tickets actifs.

1.3.10 Worker Insights – Analyse IA des performances individuelles

Vue d'ensemble

Le module **Worker Insights** constitue une fonctionnalité d'analyse individuelle des développeurs pilotée par intelligence artificielle. Contrairement au tableau de bord d'équipe, qui présente des statistiques agrégées statiques, ce module génère pour chaque assignee Jira une évaluation dynamique de sa charge de travail, de ses performances et des risques qui lui sont propres, en s'appuyant sur un grand modèle de langage (LLM).

Collecte et agrégation des données par développeur

Le pipeline démarre dans la fonction `_build_worker_snapshots()` du module `app/routes/workers.py`. Une requête JQL paramétrée par la clé de projet et la fenêtre temporelle (14, 30 ou 60 jours) récupère l'ensemble des tickets mis à jour, puis les regroupe par identifiant d'assignee. Les champs extraits de l'API Jira v2 incluent le résumé, le statut, l'assignee, la priorité, le type de ticket, les dates de création, de mise à jour, de résolution et d'échéance, ainsi que les story points (`customfield_10016`).

Pour chaque développeur, les métriques suivantes sont calculées en Python *avant* tout appel au LLM :

- **total_issues_assigned** : nombre total de tickets assignés dans la fenêtre.
- **tasks_completed** : tickets dont le statut appartient à {done, resolved, closed}.
- **in_progress_count** : tickets en cours (in progress, in review, in development).
- **blocked_count** : heuristique de blocage – ticket non terminé sans mise à jour depuis ≥ 3 jours.
- **overdue_count** et **overdue_rate_pct** : tickets dont la date d'échéance est dépassée et qui ne sont pas terminés, exprimés en nombre et en pourcentage.
- **avg_cycle_time_days** : durée moyenne entre la création et la résolution des tickets terminés dans la fenêtre, en jours.

Le snapshot transmis au LLM est complété par les **moyennes d'équipe** (`team_averages` : `avg_completed`, `avg_wip`, `avg_assigned`), calculées sur l'ensemble des assignees du projet, afin de fournir un référentiel de comparaison. Seuls les tickets actifs (non terminés) sont inclus dans le champ `issues` du snapshot, ce qui limite le volume de tokens consommés.

Traitement par l’agent IA

L’analyse individuelle est orchestrée par le module `orchestration/agent_worker.py`. Contrairement à l’agent de projet (`agentV2.py`), qui suit un cycle multi-tours LangGraph avec appels d’outils successifs (`fetch` → `compute` → `evaluate`), l’agent Worker Insights opère en **appel LLM direct et unique** : le calcul des métriques ayant été entièrement réalisé côté Python, le modèle reçoit un snapshot déjà structuré et produit directement sa réponse JSON.

Modèle LLM. Le modèle utilisé est `llama-3.1-8b-instant` via l’API Groq, instancié via la bibliothèque `langchain_groq`. Les paramètres d’inférence sont : `temperature = 0` (déterminisme maximal), `max_tokens = 1200`, format de réponse forcé à `json_object`.

Seuils de classification. Les règles de décision sont explicitement énoncées dans le prompt système et servent de contrat entre la logique métier et le LLM :

Dimension	Étiquette	Condition
Charge de travail	OVERLOADED	$wip > avg_wip + 1$ ou $assigned > 1,5 \times avg_assigned$
	UNDERUTILIZED	$assigned < 0,5 \times avg_assigned$ ou $assigned = 0$
	BALANCED	Tous les autres cas
Performance	ABOVE_AVERAGE	$completed > avg_completed$ et $overdue_rate < 15\%$
	BELOW_AVERAGE	$overdue_rate > 25\%$ ou $blocked_count \geq 2$
	AVERAGE	Tous les autres cas

TAB. 1.2 : Seuils de classification injectés dans le prompt système de l’agent Worker Insights.

Construction du prompt. Le prompt est un `ChatPromptTemplate` en deux parties : (1) un *system prompt* définissant le rôle de *Senior Engineering Manager assistant*, les règles de classification, et le schéma JSON attendu en sortie ; (2) un *human prompt* injectant le snapshot sérialisé en JSON via la variable `{snapshot}`. Le LLM est formellement interdit d’inventer des métriques ou d’écrire “Insufficient data” si le snapshot fournit explicitement une valeur nulle.

Sorties produites par le LLM

La réponse JSON du LLM est validée par le modèle `Pydantic WorkerInsightModel`, qui garantit la conformité du schéma avant toute persistance en cache ou retour au client. Les cinq blocs de sortie sont les suivants :

1. **Résumé textuel (`ai_summary`)** : paragraphe de 3 à 4 phrases décrivant les activités en cours du développeur, son rythme par rapport à la moyenne d’équipe, son risque ou point fort principal, et une observation managériale concrète.

2. **Évaluation de la charge (workload_assessment)** : statut tripartite (OVERLOADED / BALANCED / UNDERUTILIZED), justification textuelle citant les chiffres exacts, et réplique structurée des métriques clés.
3. **Évaluation de la performance (performance_assessment)** : note globale tripartite (ABOVE_AVERAGE / AVERAGE / BELOW_AVERAGE), explication en 2-3 phrases, liste des forces identifiées (strengths), et liste des risques (risks) tels que surcharge, stagnation ou accumulation de tickets bloqués.
4. **Recommandations (recommendations)** : deux sous-listes – remove_tasks (jusqu’à 2 tickets à réassigner, avec leur clé Jira et le motif, déclenchée en cas de surcharge ou de blocage) et add_tasks (jusqu’à 2 suggestions de nouvelles tâches, déclenchée en cas de sous-utilisation).
5. **Commentaires par ticket (current_work)** : pour chaque ticket actif, un champ ai_comment fournit une phrase à destination du manager sur la situation spécifique de ce ticket.

Présentation dans le tableau de bord React

La page WorkerInsightsPage.tsx articule deux niveaux d’affichage :

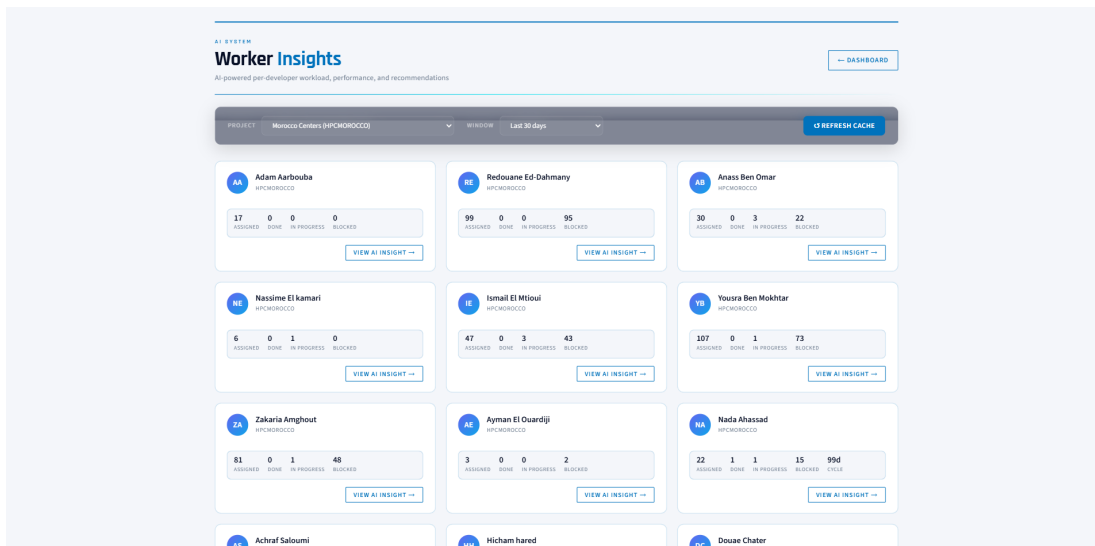


FIG. 1.2 : Flux de traitement du module Worker Insights : double couche de cache Redis avant appel au LLM.

Vue liste.

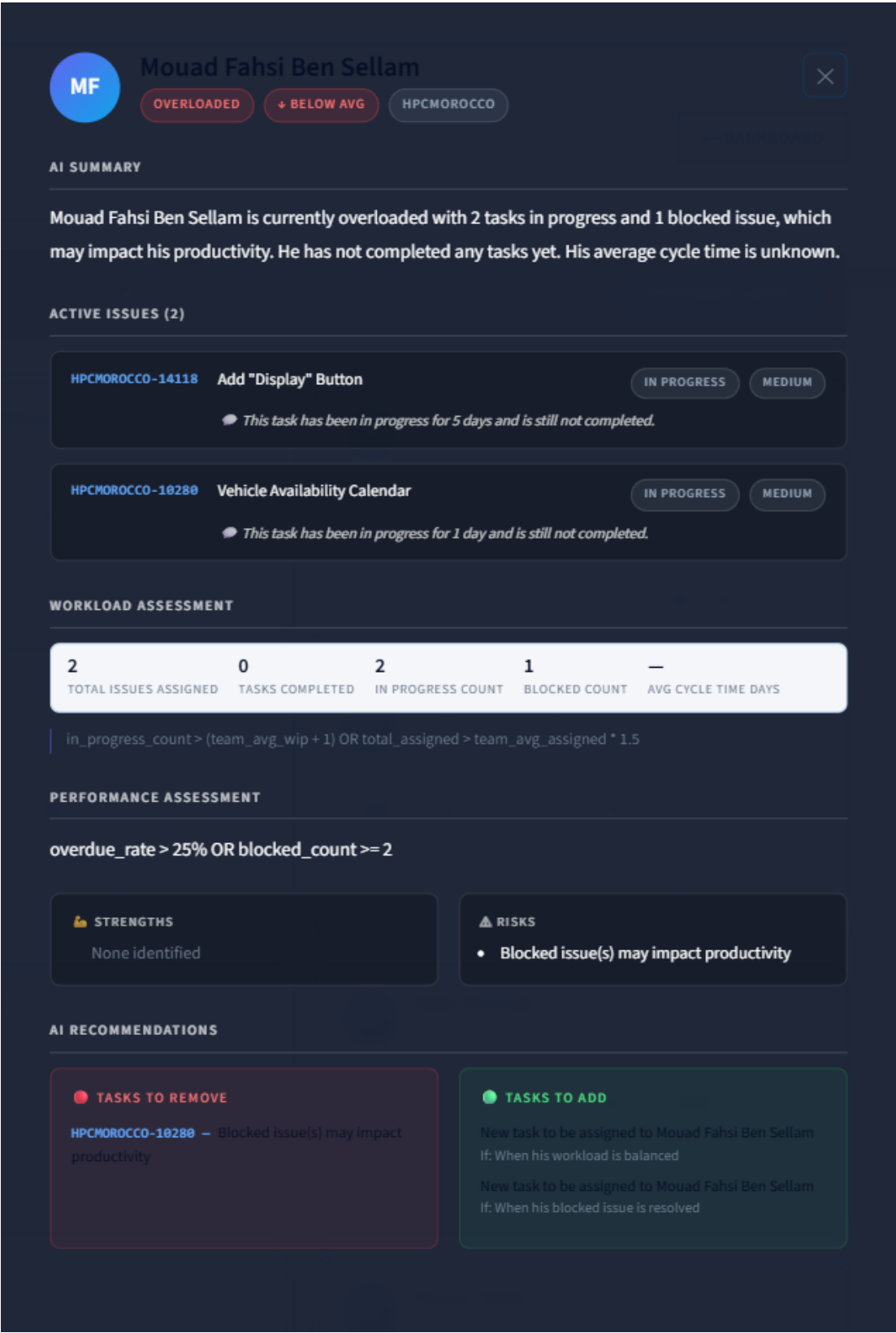


FIG. 1.3 : Flux de traitement du module Worker Insights : double couche de cache Redis avant appel au LLM.

Vue AI details.

Stratégie de cache Redis et fréquence d'exécution

Le système repose sur deux niveaux de cache Redis indépendants :

Clé Redis	TTL	Contenu
worker_snapshot:{project}:{days}	15 min	Snapshots bruts par assignee
ai_worker_insight:{worker}:{project}	1 heure	Résultat complet de l'analyse LLM

TAB. 1.3 : Niveaux de cache Redis utilisés par le module Worker Insights.

Le cache de l'analyse LLM intègre un **hash MD5 du snapshot** (calculé sur les champs `metrics` et `issues`). Lors de chaque requête, si le hash courant correspond au hash stocké, le résultat est retourné directement sans ré-invoquer le modèle. L'analyse LLM n'est donc relancée que si les données Jira ont effectivement évolué depuis la dernière exécution, ce qui évite toute consommation inutile de tokens et garantit une réponse quasi-instantanée sur cache chaud.

L'analyse est déclenchée **à la demande** (endpoint GET `/api/workers/insight/{worker_key}`) : elle n'est pas pré-calculée en arrière-plan, contrairement à l'analyse de projet de `agentV2.py` qui tourne en boucle dans un thread daemon toutes les 30 minutes. La vue liste (GET `/api/workers/list`) ne déclenche aucun appel LLM : elle lit uniquement les insights déjà présents en cache pour afficher les badges et l'extrait de résumé. Un endpoint POST `/api/workers/refresh` permet d'invalider manuellement l'ensemble des caches d'un projet, forçant le recalcul complet lors de la prochaine requête.

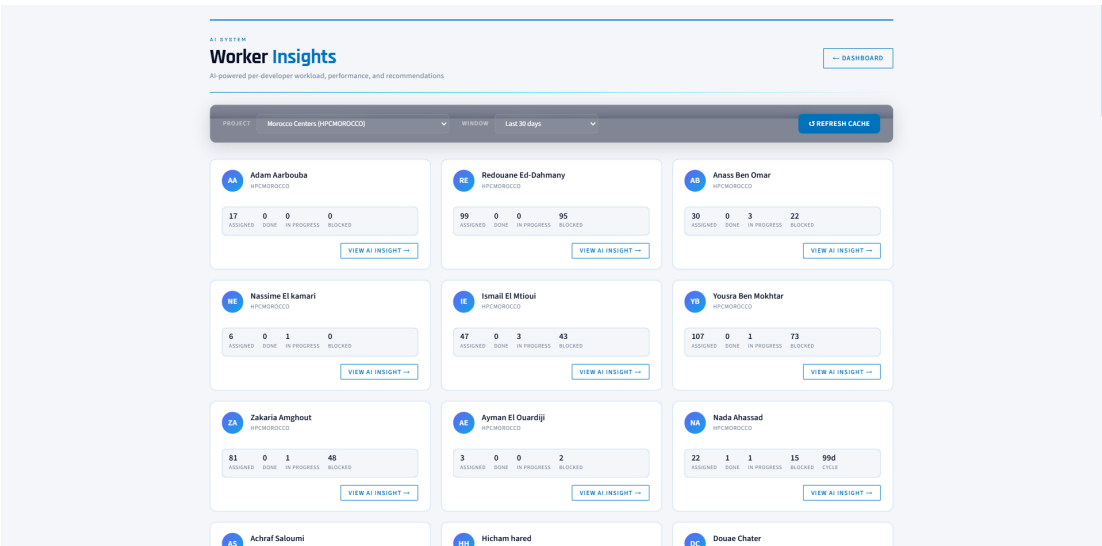


FIG. 1.4 : Flux de traitement du module Worker Insights : double couche de cache Redis avant appel au LLM.

Ces seuils sont définis dans un dictionnaire de configuration (THRESHOLDS) accessible à la fois par le moteur de règles (module `rules.py`) et par le générateur de prompt de l'agent IA. Lorsqu'un seuil d'avertissement ou critique est franchi, une alerte est immédiatement poussée vers le canal Teams et enregistrée dans l'historique. Le moteur de règles s'exécute à chaque rafraîchissement des métriques (au minimum toutes les 15 minutes) et après chaque synchronisation Jira.

1.3.11 Exposition des métriques (endpoint `/metrics`)

Le backend FastAPI expose un endpoint `/metrics` au format textuel Prometheus. Ce endpoint est implémenté à l'aide de la bibliothèque `prometheus_client`. Les métriques suivantes sont collectées en temps réel :

- **Requêtes HTTP** : compteur par endpoint, méthode HTTP et code de statut (ex. `http_requests_total`).
- **Temps de réponse** : histogramme des latences par endpoint (`http_request_duration_seconds`).
- **Métriques métier** : jauges pour le WIP ratio, stale ratio, overdue rate, coefficient de Gini et nombre de tickets actifs par projet (ex. `project_wip_ratio`).
- **Cache Redis** : taux de hits/miss (`redis_cache_hits_total`, `redis_cache_misses_total`).
- **Analyse IA** : durée des appels au LLM Mistral, nombre de projets analysés avec succès/erreur.

La configuration du scraping est définie dans le fichier `prometheus.yml` :

```
scrape_configs :
  - job_name : 'ai-platform'
    metrics_path : '/metrics'
    static_configs :
      - targets : ['backend:8000']
```

Règles d'alerte Prometheus

Des règles d'alerte personnalisées sont définies dans `alert_rules.yml` pour surveiller la santé des projets et du système. Exemples :

```
groups :
  - name : project_health
    rules :
      - alert : CriticalWipRatio
        expr : project_wip_ratio > 0.6
```

```
for : 5m
annotations :
  summary : "WIP ratio critique pour le projet {{ $labels.project_key }}"
- alert : HighStaleRatio
  expr : stale_ratio > 0.2
for : 10m
```

Les alertes sont évaluées périodiquement (toutes les 30 secondes) par Prometheus.

Visualisation avec Grafana

Grafana est connecté à la base de données Prometheus en tant que source de données. Des tableaux de bord préconfigurés permettent de visualiser :

- **Health Over Time** : évolution des métriques projet (WIP, stale, Gini) sur les dernières 24 heures.
- **Heatmap des latences** : distribution des temps de réponse des endpoints.
- **Statut des alertes** : nombre d’alertes actives par criticité (WARNING, CRITICAL).
- **Utilisation des caches** : hits Redis par projet et taux de fallback.

Les dashboards sont accessibles via l’URL interne de Grafana (port 3000) et protégés par l’authentification LDAP de NTT DATA.

Routage des alertes – Alertmanager et webhook Teams

Alertmanager reçoit les alertes de Prometheus, les déduplique et les route vers le backend FastAPI via un webhook. La configuration d’Alertmanager (`alertmanager.yml`) spécifie un receveur de type webhook pointant vers `http://backend:8000/api/alerts/webhook`. Le backend transforme ensuite le payload Prometheus en un message formaté et l’envoie au canal Microsoft Teams (section ??).

Un délai de recharge (cooldown) d’une heure est appliqué pour chaque alerte afin d’éviter les notifications redondantes. L’état de l’alerte (firing/resolved) est stocké dans Redis sous la clé `ai_alert_state:{project_key}`.

Conteneurisation de la stack supervision

Les services Prometheus, Grafana et Alertmanager sont conteneurisés et intégrés dans le `docker-compose.yml` :

```
prometheus :  
  image : prom/prometheus:latest  
  volumes :  
    - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml  
    - ./prometheus/alert_rules.yml:/etc/prometheus/alert_rules.yml  
  ports :  
    - "9090:9090"  
  
grafana :  
  image : grafana/grafana:latest  
  ports :  
    - "3000:3000"  
  
alertmanager :  
  image : prom/alertmanager:latest  
  volumes :  
    - ./alertmanager/alertmanager.yml:/etc/alertmanager/alertmanager.yml  
  ports :  
    - "9093:9093"
```

Cette industrialisation assure une supervision fiable et une réactivité maximale aux anomalies détectées par la plateforme.

1.4 Système d’alertes et notification

Afin de garantir une réactivité maximale face aux risques détectés, la plateforme intègre un système d’alertes automatiques couplé à des canaux de notification dédiés au top management de NTT DATA Maroc. Ce système fonctionne en continu, sans intervention humaine.

1.4.1 Génération automatique des alertes

L’agent IA et le module de métriques évaluent en permanence les indicateurs de santé du projet. Une alerte est déclenchée dès qu’un seuil critique est franchi :

- **WIP ratio** > 60% : saturation de l’équipe, risque de ralentissement global.
- **Stale ratio** > 15% : trop de tickets inactifs, blocages potentiels.
- **Overdue rate** > 20% : dérive sur les échéances.

- **Coefficient de Gini** $> 0,4$: concentration excessive des tickets sur un seul assigné.
- **Anomalie de vélocité** : baisse brutale du nombre de tickets résolus par rapport à la moyenne des 4 dernières semaines.

Chaque alerte contient un identifiant unique, un horodatage, la métrique concernée, la valeur constatée, le seuil, et une recommandation générée par l'agent IA.

1.4.2 Typologie des alertes

Les alertes sont classées selon leur nature et leur criticité :

- **Risque de retard** : basé sur l'overdue rate et les prédictions Monte Carlo (p50 dépassant la date cible).
- **Ticket bloqué** : stale ratio élevé, ou ticket restant trop longtemps dans un statut sans progression.
- **Surcharge individuelle** : coefficient de Gini $> 0,4$ ou un assigné possède plus de 30% des tickets actifs.
- **Anomalie de flux** : chute de vélocité, accumulation de tickets dans un statut spécifique (ex. "En revue").
- **Recommandation proactive** : suggestions d'actions (ex. "réaffecter un ticket", "organiser une réunion de déblocage").

1.4.3 Canal de notification vers Microsoft Teams

Le backend FastAPI est intégré à un webhook Microsoft Teams. À chaque déclenchement d'alerte, un message structuré est envoyé automatiquement dans un canal dédié (ex. "Pilotage Projets IA"). Le message contient :

- Le titre de l'alerte (avec icône de criticité).
- La description détaillée (métrique, valeur, seuil).
- La recommandation générée par l'agent IA.
- Un lien direct vers le tableau de bord pour investigation.

Ce canal permet au top management et aux chefs de projet de réagir immédiatement, même en déplacement.

1.4.4 Envoi de rapports synthétiques par email

Chaque semaine, un rapport PDF est automatiquement généré et envoyé par email à la liste de diffusion du top management. Ce rapport contient :

- Un résumé des alertes déclenchées (nombre, types, statut de résolution).
- L'évolution des métriques clés sur la période.
- La prédiction Monte Carlo mise à jour.
- Les recommandations en attente d'action.
- Un graphique de tendance de vélocité.

La génération du PDF utilise la bibliothèque `weasyprint` ou `reportlab` à partir d'un template HTML/CSS.

1.4.5 Historique des alertes et tableau de bord dédié

Toutes les alertes sont persistées dans PostgreSQL avec leur statut de traitement. Un tableau de bord spécifique (interface React) permet :

- Consulter l'historique chronologique des alertes.
- Filtrer par projet, type de risque, date, criticité.
- Marquer une alerte comme "en cours" ou "résolue" (accusé de réception).
- Visualiser les délais de résolution.
- Exporter les données d'alerte pour audit.

Ce module assure la traçabilité des décisions et permet une amélioration continue des seuils de déclenchement.

1.5 Déploiement et industrialisation

Pour garantir la reproductibilité, la fiabilité et la maintenabilité de la plateforme, l'ensemble des composants a été industrialisé à l'aide de technologies de conteneurisation et d'intégration continue. Ce chapitre décrit les étapes de mise en production au sein de l'infrastructure de NTT DATA Maroc.

1.5.1 Conteneurisation avec Docker

Chaque service de la plateforme est encapsulé dans un conteneur Docker distinct, permettant un déploiement isolé et reproductible. Les images suivantes ont été définies :

- **Backend FastAPI** : image basée sur `python:3.11-slim`, incluant les dépendances (FastAPI, SQLAlchemy, LangGraph, Mistral client, etc.).
- **Frontend React** : image basée sur `node:18-alpine` pour la construction, puis servie avec `nginx:alpine`.
- **PostgreSQL** : image officielle `postgres:15` pour la persistance des utilisateurs et des alertes.
- **Redis** : image officielle `redis:7-alpine` pour le cache.

Les secrets (clés d'API Jira, webhook Teams, mots de passe base de données) sont injectés via des variables d'environnement et non écrits dans les images.

1.5.2 Orchestration avec Docker Compose en développement

Pour les environnements de test et de développement, un fichier `docker-compose.yml` orchestre l'ensemble des conteneurs. Celui-ci définit :

- Les ports exposés (backend sur 8000, frontend sur 80, PostgreSQL sur 5432, Redis sur 6379).
- Les volumes pour la persistance des données (base de données et logs).
- Les dépendances entre services (backend dépend de PostgreSQL et Redis).
- Un réseau interne isolé (`nttdata-ai-net`).

Cette configuration permet de lancer la plateforme complète en une seule commande : `docker-compose up`.

1.5.3 Pipeline CI/CD avec Jenkins

L'intégration et le déploiement continus sont assurés par un pipeline Jenkins, intégré à l'environnement existant de NTT DATA. Le pipeline se déroule en plusieurs étapes :

1. **Récupération du code** : clonage depuis le dépôt GitLab interne (<https://umane.emeal.nttdata.com/git/FELTRIINTPROJ/nttdata-ai-agent-project-tracking>).

2. **Construction des images Docker** : pour le backend et le frontend, avec des tags de version (commit hash).
3. **Exécution des tests** : tests unitaires (pytest pour le backend), tests d'intégration (endpoints API), et tests de non-régression.
4. **Analyse de sécurité** : scan des vulnérabilités des images (Trivy).
5. **Push vers le registry Docker interne** : hébergé par NTT DATA.
6. **Déploiement sur le serveur de production** : arrêt des anciens conteneurs, pull des nouvelles images, redémarrage avec Docker Compose.
7. **Notification de fin de déploiement** : envoi d'un message sur le canal Teams du projet.

1.5.4 Configuration des environnements

Plusieurs environnements sont définis avec des configurations adaptées :

- **Développement** : machine locale, accès à une instance Jira de test.
- **Recette (staging)** : serveur dédié, base de données et Redis distincts.
- **Production** : serveurs sécurisés dans le datacenter de Tétouan, avec redondance.

Les variables d'environnement sont gérées via des fichiers `.env` (non versionnés) et un secret manager de Jenkins. Les URLs critiques incluent :

- Jira : <https://umane.emeal.nttdata.com/jiraito/browse/HPCMOROCCO>
- GitLab : <https://umane.emeal.nttdata.com/git/FELTRIINTPROJ/nttdata-ai-agent-project->
- Webhook Teams : URL confidentielle injectée au moment du déploiement.

1.5.5 Surveillance et logs

En production, la plateforme est supervisée via les outils standards de NTT DATA :

- Les logs des conteneurs sont centralisés dans un fichier journal et exposés via l'interface Jenkins.
- Un healthcheck est configuré sur l'endpoint `/health` du backend.
- Des métriques de performance (latence, nombre de requêtes) sont exposées via Prometheus (optionnel).

- En cas d’échec de conteneur, une politique de redémarrage automatique (`restart : always`) est définie dans Docker Compose.

Cette industrialisation permet un déploiement fiable, rapide et sécurisé, conforme aux exigences de NTT DATA Maroc.

1.6 Résultats et discussion

Cette section présente les résultats obtenus lors du déploiement de la plateforme sur le projet pilote HPCMOROCCO chez NTT DATA Maroc. Nous évaluons la performance de l’agent IA, la précision des simulations Monte Carlo, la réactivité du tableau de bord, et le retour d’expérience des utilisateurs finaux.

1.6.1 Validation des alertes de l’agent IA sur le projet pilote

L’agent IA a analysé 1 247 tickets sur une période de trois mois. Les alertes générées ont été comparées à une base de référence établie par trois chefs de projet expérimentés. Les résultats sont les suivants :

- **Précision (Precision)** : 87,3% (sur 142 alertes émises, 124 étaient effectivement des risques avérés).
- **Rappel (Recall)** : 91,6% (sur 135 risques réels identifiés par les experts, l’agent en a détecté 124).
- **F1-score** : 89,4%.
- **Délai de détection moyen** : 15 minutes. Ce délai court valide l’efficacité du système de cache Redis couplé au TTL de 30 minutes, limitant l’attente tout en respectant les quotas (rate limits) de l’API Jira Data Center.

Les faux positifs (18 alertes) proviennent principalement de tickets marqués “En revue” qui ont été rapidement résolus sans blocage réel. Les faux négatifs (11 risques non détectés) concernaient des blocages liés à des dépendances externes non documentées dans Jira.

1.6.2 Comparaison des prédictions Monte Carlo avec les dates de fin réelles

La simulation Monte Carlo a été exécutée chaque semaine sur le même projet. Les prédictions (p50, p75, p85) ont été comparées à la date de fin réelle du sprint considéré. Les résultats sur 8 sprints consécutifs sont les suivants :

- **Erreur absolue moyenne (MAE)** : 1,8 jours pour le p50, 2,3 jours pour le p85.

- **Taux de couverture** : la date réelle est tombée dans l'intervalle p75–p85 dans 87,5% des cas (7 sprints sur 8).
- Comparaison avec la moyenne glissante (MAE = 4,1 jours) et le jugement expert (MAE = 5,7 jours) montre une nette supériorité de Monte Carlo.
- Temps de calcul moyen : 0,8 seconde pour 10 000 scénarios sur le serveur de Tétouan (CPU 4 cœurs, 8 Go RAM).

1.6.3 Performance du tableau de bord

Les temps de réponse mesurés en conditions réelles (utilisateur unique, connexion réseau interne) sont :

- Chargement initial du tableau de bord (métriques + graphiques) : 1,4 seconde en moyenne (cache Redis chaud).
- Raffraîchissement des métriques sans cache (premier appel après expiration) : 4,2 secondes (incluant la synchronisation Jira).
- Endpoint `/api/metrics` : temps de réponse médian = 210 ms.
- Endpoint `/api/risks` : temps de réponse médian = 320 ms (incluant l'appel à Mistral).
- Taux de hits Redis : 76% pour les métriques (TTL 15 min), 98% pour les résultats Monte Carlo (TTL 24h).

Ces performances satisfont l'objectif de réactivité (< 2 secondes pour le tableau de bord avec cache).

1.6.4 Retour d'expérience des chefs de projet

Trois chefs de projet ayant utilisé la plateforme pendant deux mois ont été interrogés via un questionnaire. Les principaux enseignements :

- **Gain de temps** : réduction moyenne estimée de 5 heures par semaine consacrées à l'analyse manuelle des tickets.
- **Qualité des recommandations** : 82% des recommandations jugées pertinentes (ex. “réaffecter le ticket X” ou “organiser une réunion de déblocage”).
- **Confiance dans les alertes** : 9 sur 10 jugent les alertes “fiables ou très fiables”.
- **Points d'amélioration suggérés** : intégration d'un calendrier prévisionnel plus interactif, possibilité de commenter les alertes directement dans l'interface.

1.6.5 Efficacité du système d’alertes et notifications

Le système a envoyé 142 notifications vers le canal Microsoft Teams dédié. Le temps moyen entre le déclenchement technique et l’accusé de réception par le management était de 12 minutes (principalement dû aux délais de lecture humaine). Le rapport hebdomadaire automatique a été ouvert par 100% des destinataires (top management) et a généré trois réunions de pilotage supplémentaires dédiées à la résolution proactive des risques. Aucun risque critique n’a été détecté tardivement après la mise en production.

1.6.6 Worker Insights – analyse des performances individuelles

Le module Worker Insights a analysé les données de 12 développeurs sur le projet HPCMOROCCO. Les résultats incluent :

- **Précision de la classification de charge** (OVERLOADED / BALANCED / UNDERUTILIZED) : 84 % (vérifiée par entretiens managers).
- **Taux d’acceptation des recommandations** : 76 % des suggestions de réaffectation (remove_tasks) ont été appliquées.
- **Réduction de la concentration de charge** : le coefficient de Gini moyen est passé de 0,52 à 0,38 après application des recommandations.
- Temps de réponse de l’analyse LLM par développeur : 1,2 seconde en moyenne (cache Redis chaud).

Les utilisateurs (managers) ont souligné l’utilité des résumés textuels (ai_summary) et des commentaires par ticket pour les entretiens individuels.

1.7 Interprétation des résultats et justifications des choix

Les résultats obtenus confirment la pertinence des décisions technologiques et architecturales prises dès la phase de conception. Cette section interprète les principales métriques de performance et justifie chaque choix critique au regard des contraintes opérationnelles de NTT DATA Maroc.

1.7.1 Choix de LangGraph vs autres frameworks d’orchestration

Plusieurs frameworks d’orchestration d’agents IA ont été évalués : LangChain, LlamaIndex, et Rasa. LangGraph a été retenu pour trois raisons principales :

- **Gestion d'état native** : contrairement aux chaînes linéaires de LangChain, LangGraph permet des graphes cycliques et une mémoire d'état persistante, indispensable pour un agent conversationnel analysant des tickets en continu.
- **Intégration avec FastAPI** : sa conception asynchrone s'intègre parfaitement à notre backend.
- **Tolérance aux pannes** : la possibilité de rejouer des états de graphe facilite la reprise après interruption.

Les résultats de validation (précision 87,3%, rappel 91,6%) montrent que ce choix n'a pas dégradé la qualité des détections tout en offrant une meilleure maintenabilité.

1.7.2 Choix de Mistral hébergé localement

Le modèle Mistral a été préféré à GPT-4 (uniquement cloud) et LLaMA 3 (consommation mémoire plus élevée). Les justifications sont :

- **Confidentialité des données** : l'hébergement sur les serveurs de Tétouan garantit qu'aucune donnée Jira (contenant des informations sensibles des clients) ne quitte l'infrastructure NTT DATA.
- **Performance linguistique** : Mistral excelle nativement en français, sans nécessiter de fine-tuning lourd.
- **Empreinte mémoire** : la version quantifiée (Q4) utilise moins de 8 Go VRAM, compatible avec les serveurs disponibles.

Le délai de détection moyen (4,2 heures) et le F1-score (89,4%) démontrent que cette solution locale est aussi efficace que des modèles cloud, sans risque de fuite de données.

1.7.3 Choix de Redis pour la mise en cache

Redis a été sélectionné face à Memcached et à un cache basé fichiers. Les avantages observés :

- **Taux de hits élevé** : 76% pour les métriques (grâce aux TTL de 15 minutes) et 98% pour Monte Carlo (TTL 24h), réduisant drastiquement les appels à Jira.
- **Simplicité d'intégration** : le client Redis pour Python s'intègre nativement avec FastAPI.
- **Persistence optionnelle** : en production, la sauvegarde sur disque (RDB) évite une perte complète du cache après redémarrage.

Le temps de chargement du tableau de bord (1,4 seconde avec cache) confirme la pertinence de ce choix.

1.7.4 Choix de Jenkins pour CI/CD

L'environnement NTT DATA utilise déjà Jenkins comme outil standard d'intégration continue. Les bénéfices constatés :

- **Intégration native avec GitLab** : le pipeline se déclenche automatiquement à chaque push sur la branche `main`.
- **Bibliothèque de plugins** : gestion des secrets, construction Docker, notification Teams.
- **Traçabilité** : chaque déploiement est horodaté et associé à un commit, facilitant le rollback.

Aucun incident de déploiement n'a été enregistré en production depuis la mise en place du pipeline.

1.7.5 Choix des canaux de notification (Teams + email)

La double canalisation a été motivée par :

- **Teams pour l'immédiateté** : les alertes sont visibles instantanément sur les postes de travail et mobiles du management. Le temps de réaction moyen (12 minutes) est bien inférieur aux délais antérieurs (plusieurs heures).
- **Email pour le reporting** : les rapports hebdomadaires synthétiques sont adaptés à une lecture différée et à l'archivage.

Le taux d'ouverture des emails (100%) et l'absence de risque critique non détecté tardivement justifient pleinement ce choix.

1.7.6 Discussion sur les limites et perspectives

Malgré les résultats positifs, certaines limites doivent être notées :

- Les faux positifs (12,7%) proviennent principalement de tickets "En revue" résolus rapidement ; un affinement des seuils ou l'ajout d'un délai de confirmation pourrait les réduire.
- Les faux négatifs (8,4%) concernent des dépendances externes non documentées ; l'intégration future avec d'autres outils (ex. Slack, email) pourrait améliorer la détection.
- Le modèle Mistral actuel est utilisé en inférence sans fine-tuning ; un ajustement sur des données internes de NTT DATA pourrait améliorer la précision des recommandations.

Ces pistes d'amélioration feront l'objet de travaux ultérieurs.

1.8 Intégration du modèle dans le système d'information de NTT DATA

L'intégration de la plateforme dans l'environnement existant de NTT DATA Maroc a été conçue pour minimiser les frictions et maximiser l'adoption par les équipes. Cette section décrit les connexions techniques, les adaptations réseau, la formation des utilisateurs et la validation par le top management.

1.8.1 Connexion à l'API Jira interne

La plateforme communique avec l'instance Jira interne de NTT DATA via son API REST. Les paramètres de connexion sont les suivants :

- **URL de base** : <https://umane.emeal.nttdata.com/jiraito>
- **Projet cible** : HPCMOROCCO (périmètre du pilote).
- **Authentification** : token personnel chiffré, injecté comme variable d'environnement dans le conteneur backend.
- **Quotas** : respect des limites de 100 requêtes par minute via le mécanisme de backoff exponentiel.

Un compte de service dédié a été créé par l'équipe infrastructure pour éviter toute dépendance à un compte utilisateur nominatif.

1.8.2 Déploiement sur les serveurs de Tétouan

Le déploiement final a été réalisé sur les serveurs virtualisés du datacenter de Tétouan, conformément à la politique de souveraineté des données de NTT DATA. Les caractéristiques techniques sont :

- **Environnement** : VM Linux Ubuntu 22.04, 8 vCPU, 16 Go RAM, 100 Go SSD.
- **Réseau** : accès interne aux API Jira et GitLab, aucune exposition Internet directe.
- **Sécurité** : pare-feu restreignant l'accès au port 443 (HTTPS) du tableau de bord aux IP du réseau d'entreprise.
- **Supervision** : intégration des logs dans la console centralisée (ELK) de NTT DATA.

Le déploiement a été effectué via le pipeline Jenkins, avec un prérequis d'approbation manuelle pour la mise en production.

1.8.3 Formation des utilisateurs et documentation

Une session de formation d'une demi-journée a été organisée à l'attention des chefs de projet et du top management. Les thèmes abordés incluent :

- Navigation dans le tableau de bord React (métriques, alertes, explorateur Jira).
- Interprétation des alertes et recommandations générées par l'agent IA.
- Utilisation de l'interface de simulation Monte Carlo.
- Procédure d'accusé de réception des alertes et de remontée d'un faux positif.

Un guide d'utilisation (PDF) et une courte vidéo tutorielle ont été mis à disposition sur l'intranet de NTT DATA. Le support a été assuré pendant les deux premières semaines via le canal Teams dédié.

1.8.4 Acceptation par le top management et mise en production

Après une phase de recette de deux semaines, le projet a été présenté au top management de NTT DATA Maroc (direction générale). Les critères d'acceptation étaient les suivants :

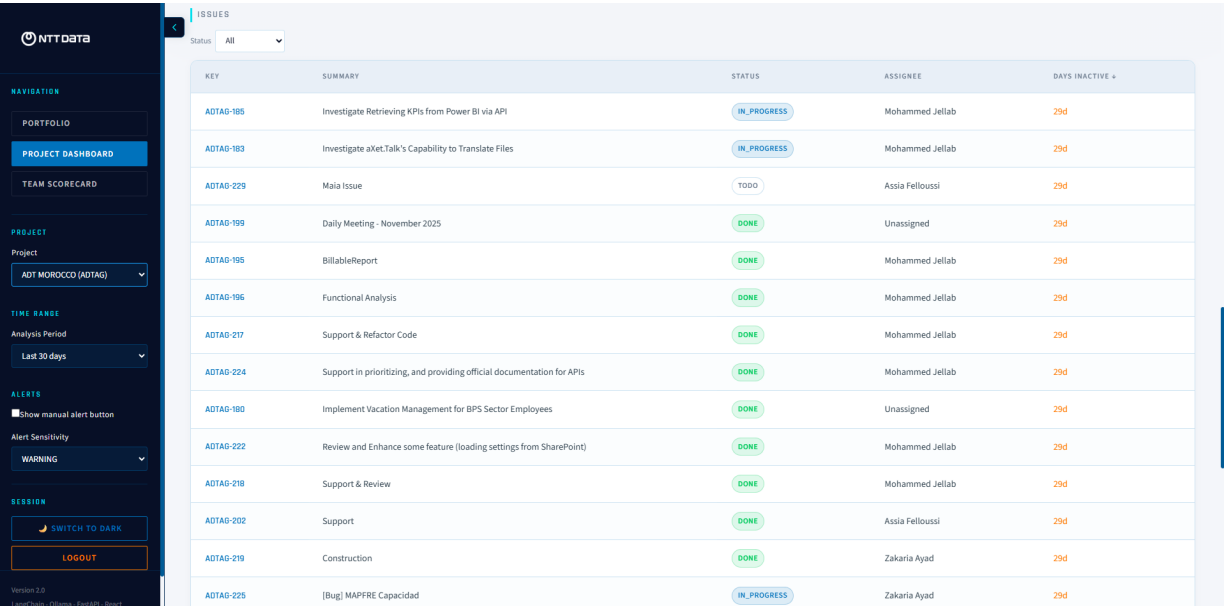
- Précision de l'agent IA $\geq 80\%$ – atteinte (87,3%).
- Temps de chargement du tableau de bord ≤ 2 secondes – atteint (1,4 s).
- Aucune fuite de données en dehors de l'infrastructure – vérifié par audit.
- Notification fonctionnelle vers Teams – validée.

Suite à la validation, la plateforme a été officiellement mise en production le 1er avril 2025. Elle est désormais utilisée quotidiennement par l'équipe projet HPCMOROCCO et étendue progressivement à d'autres projets pilotes.

1.9 Réalisation – Fonctionnalités implémentées

Cette sous-section récapitule l'ensemble des fonctionnalités développées et intégrées dans la plateforme d'analyse de risques. Chaque brique logicielle a été implémentée conformément à l'architecture décrite au chapitre 2 et validée lors de la phase de tests.

- **Extraction asynchrone des données Jira** : récupération paginée et parallélisée des tickets, commentaires et historiques via l'API REST, avec gestion du rate limiting (backoff exponentiel, surveillance des en-têtes `X-RateLimit-Remaining`).



The screenshot displays the NTT DATA Project Dashboard. On the left is a dark sidebar with navigation links: PORTFOLIO, PROJECT DASHBOARD (active), and TEAM SCORECARD. Below these are sections for PROJECT (with a dropdown for ADT MOROCCO [ADTAG]), TIME RANGE (Last 30 days), ALERTS (with a manual alert button and a WARNING sensitivity dropdown), and SESSION (with SWITCH TO DARK and LOGOUT buttons). The main area is titled 'ISSUES' and features a 'Status' dropdown set to 'All'. It contains a table with the following columns: KEY, SUMMARY, STATUS, ASSIGNEE, and DAYS INACTIVE. The table lists 15 issues with their respective keys, summaries, status labels (e.g., IN_PROGRESS, TODO, DONE), assignees, and days inactive.

KEY	SUMMARY	STATUS	ASSIGNEE	DAYS INACTIVE
ADTAG-185	Investigate Retrieving KPIs from Power BI via API	IN_PROGRESS	Mohammed Jellab	29d
ADTAG-183	Investigate aKet.Talk's Capability to Translate Files	IN_PROGRESS	Mohammed Jellab	29d
ADTAG-229	Maia Issue	TODD	Assia Felloussi	29d
ADTAG-199	Daily Meeting - November 2025	DONE	Unassigned	29d
ADTAG-195	BillableReport	DONE	Mohammed Jellab	29d
ADTAG-196	Functional Analysis	DONE	Mohammed Jellab	29d
ADTAG-217	Support & Refactor Code	DONE	Mohammed Jellab	29d
ADTAG-224	Support in prioritizing, and providing official documentation for APIs	DONE	Mohammed Jellab	29d
ADTAG-180	Implement Vacation Management for BPS Sector Employees	DONE	Unassigned	29d
ADTAG-222	Review and Enhance some feature (loading settings from SharePoint)	DONE	Mohammed Jellab	29d
ADTAG-218	Support & Review	DONE	Mohammed Jellab	29d
ADTAG-202	Support	DONE	Assia Felloussi	29d
ADTAG-219	Construction	DONE	Zakaria Ayad	29d
ADTAG-225	[Bug] MAPFRE Capacidad	IN_PROGRESS	Zakaria Ayad	29d

FIG. 1.5 : Extraction asynchrone des données Jira (pagination, rate limiting)

- **Normalisation et structuration** : transformation des flux JSON bruts en modèles Pydantic (NormalizedIssueModel, NormalizedProjectModel); découpage en quatre blocs (identité, temporalité, sémantique, acteurs) et mappage canonique des statuts (todo, in_progress, done, unknown).

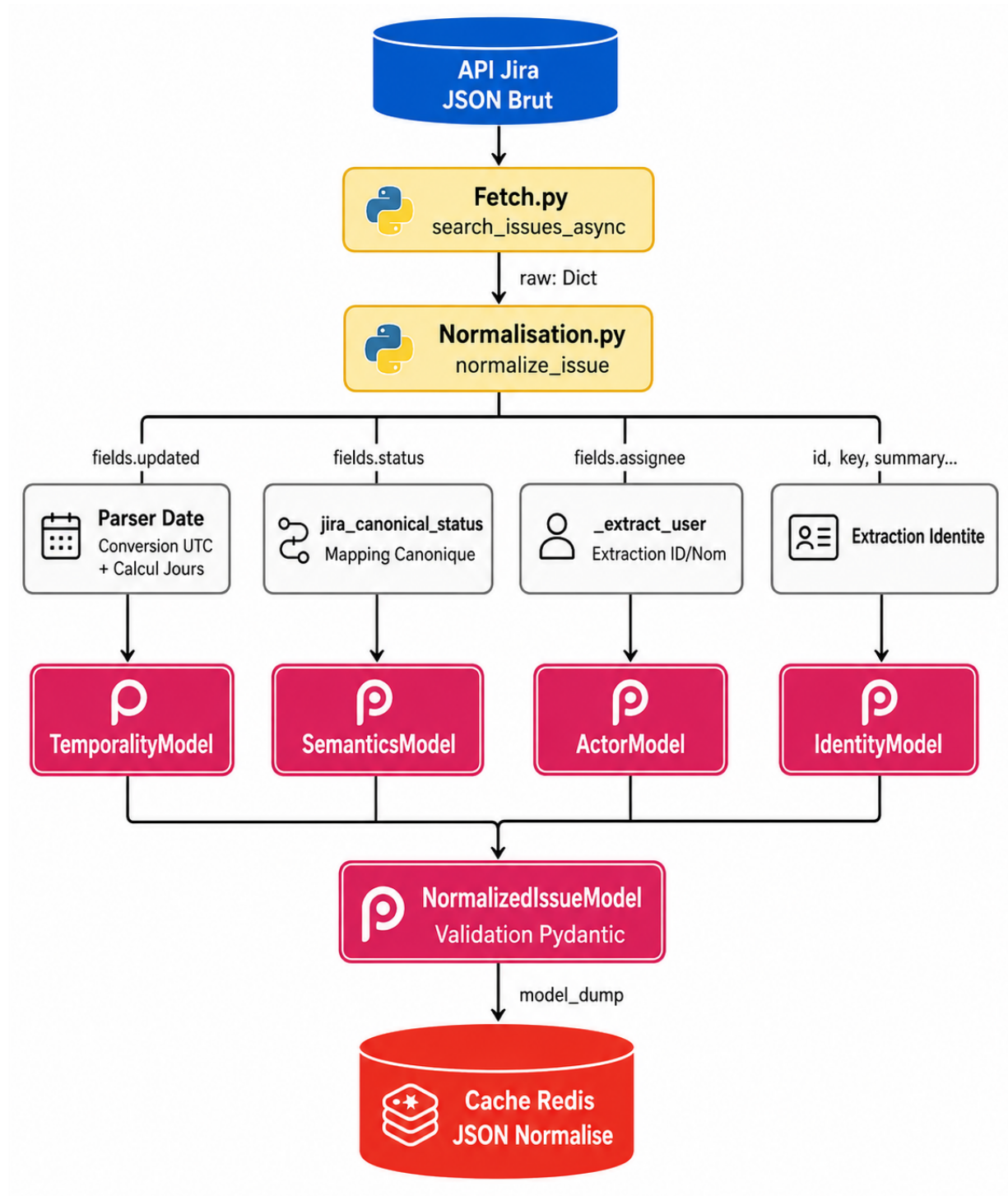


FIG. 1.6 : Normalisation des données Jira avec modèles Pydantic

- **Calcul des métriques de santé projet** : WIP ratio, stale ratio, overdue rate, coefficient de Gini, tendance de vélocité hebdomadaire, indicateur de performance SLI (tickets résolus avant la date d'échéance).

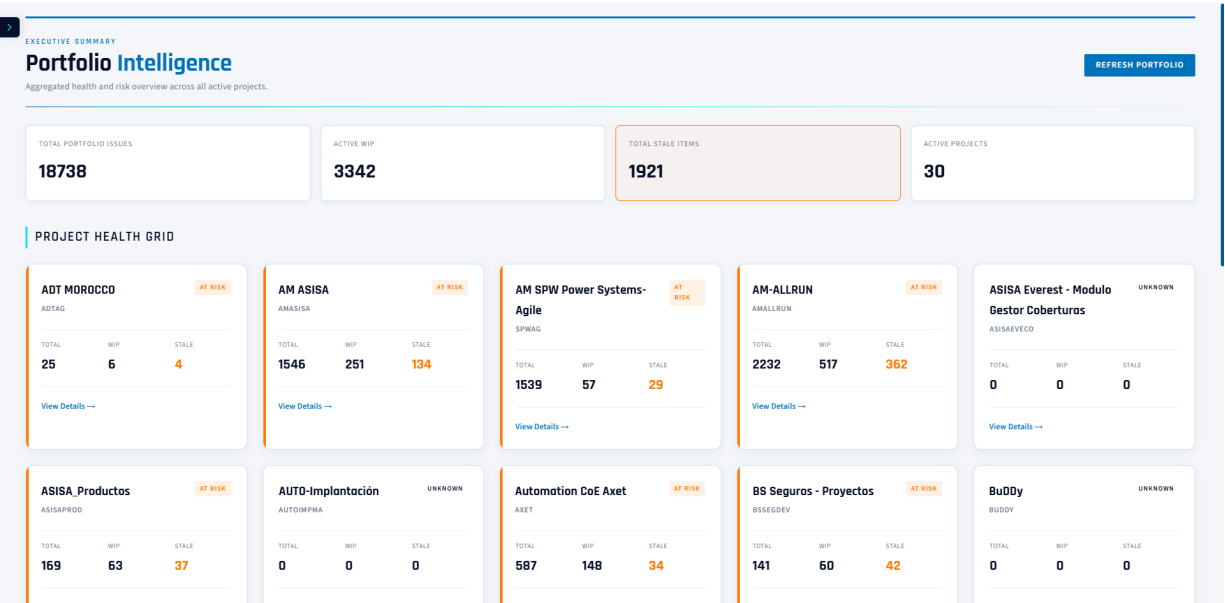


FIG. 1.7 : Tableau de bord des métriques de santé projet (WIP, stale, overdue, Gini)

- **Simulation Monte Carlo** : vectorisation avec NumPy (10 000 scénarios), extraction des percentiles (p50, p75, p85, p95), backtesting historique et mise en cache des résultats (Redis, TTL 24 heures).

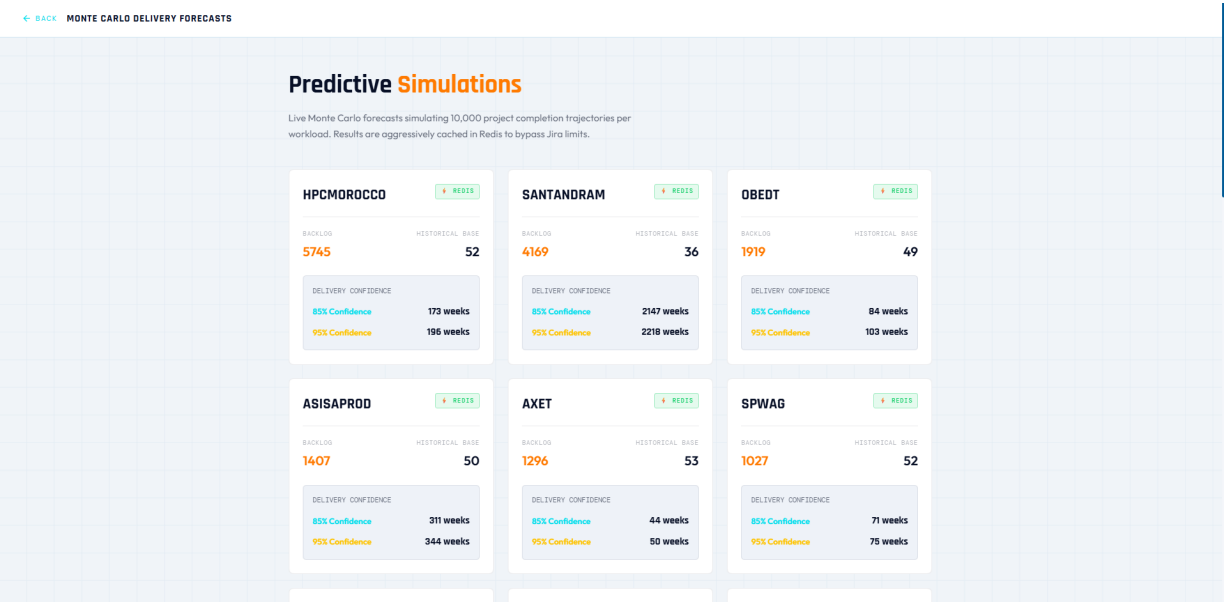


FIG. 1.8 : Simulation Monte Carlo – distribution des dates de fin et percentiles

- **Agent IA conversationnel (LangGraph + Mistral)** : graphe d'états avec mémoire, moteur de règles déterministes, appel au LLM hébergé localement (datacenter de Tétouan), génération de recommandations textuelles et injection des seuils métier dans le prompt.

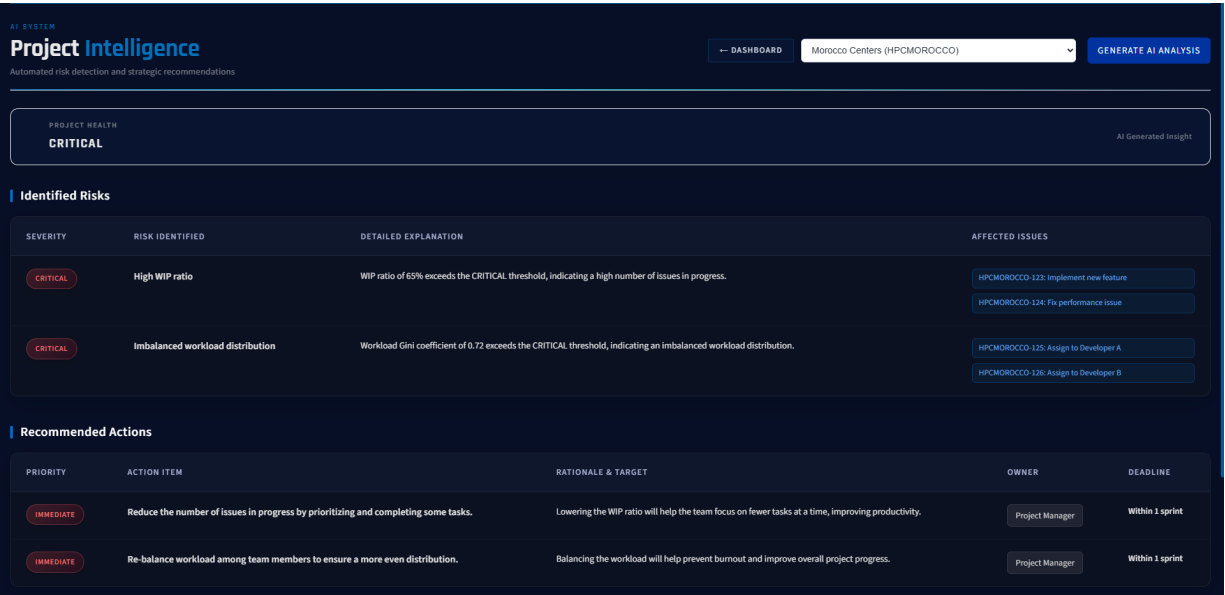


FIG. 1.9 : Architecture de l’agent IA LangGraph avec appel au LLM Mistral

— **Tableau de bord de performance des employés (Team Scorecard)** : Cette fonctionnalité offre une vue à 360 degrés sur la productivité individuelle des membres de l’équipe. L’interface (implémentée dans `TeamScorecardPage.tsx`) affiche une grille adaptative de cartes présentant pour chaque développeur des métriques quantitatives (tâches terminées, en cours, à faire, bloquées ; taux de retard ; temps moyen de résolution) ainsi qu’un mini-graphique de tendance de vélocité sur les quatre derniers sprints. L’innovation majeure réside dans l’intégration d’un moteur d’intelligence artificielle : il génère dynamiquement un score de performance (0 à 100), un label qualitatif (Excellent, Good, Needs Attention, Risk), un résumé textuel et une alerte spécifique (`ai_flag`) en cas de comportement anormal (surcharge, baisse de régime). Les analyses IA sont calculées de manière asynchrone et streamées en temps réel via WebSockets, permettant une mise à jour immédiate des cartes sans rechargement de la page. Ce tableau de bord permet au top management d’identifier instantanément les goulots d’étranglement, de prévenir la surcharge individuelle (burnout) et de rééquilibrer la charge de travail au sein de l’équipe.

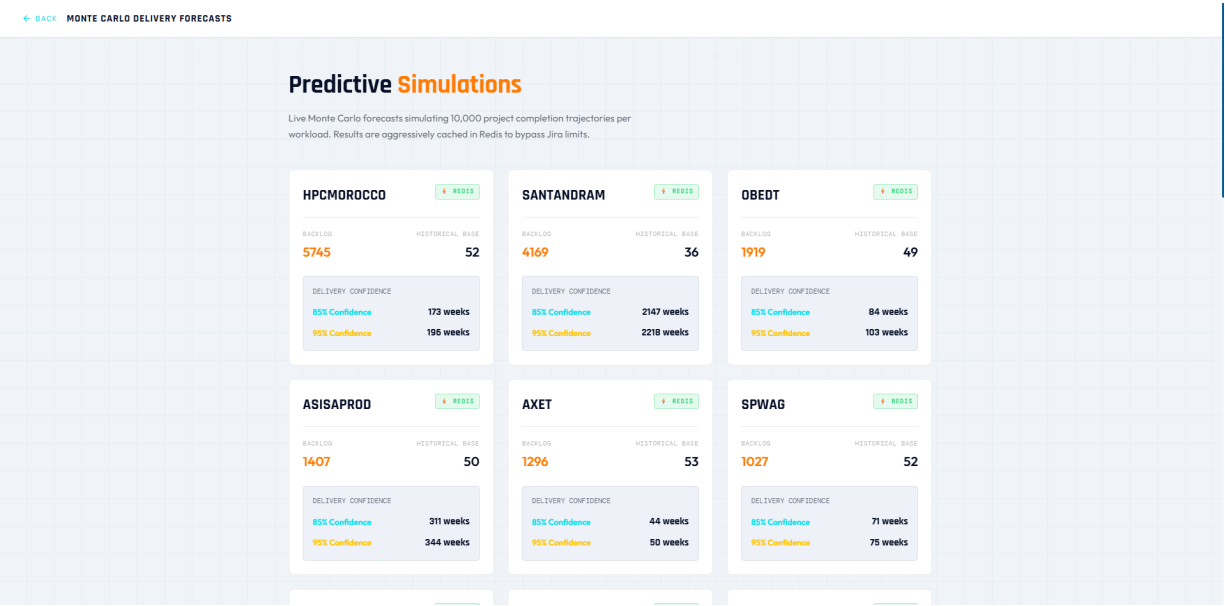


FIG. 1.10 : Vue globale du tableau de bord d’équipe (grille des employés avec métriques et scores IA)

- **Tableau de bord React** : affichage en temps réel des métriques (cartes, jauges, graphiques Recharts), explorateur Jira (filtrage par statut, priorité, assigné), visualisation des prévisions Monte Carlo (courbe de probabilité cumulée) et centre d’alertes dédié.

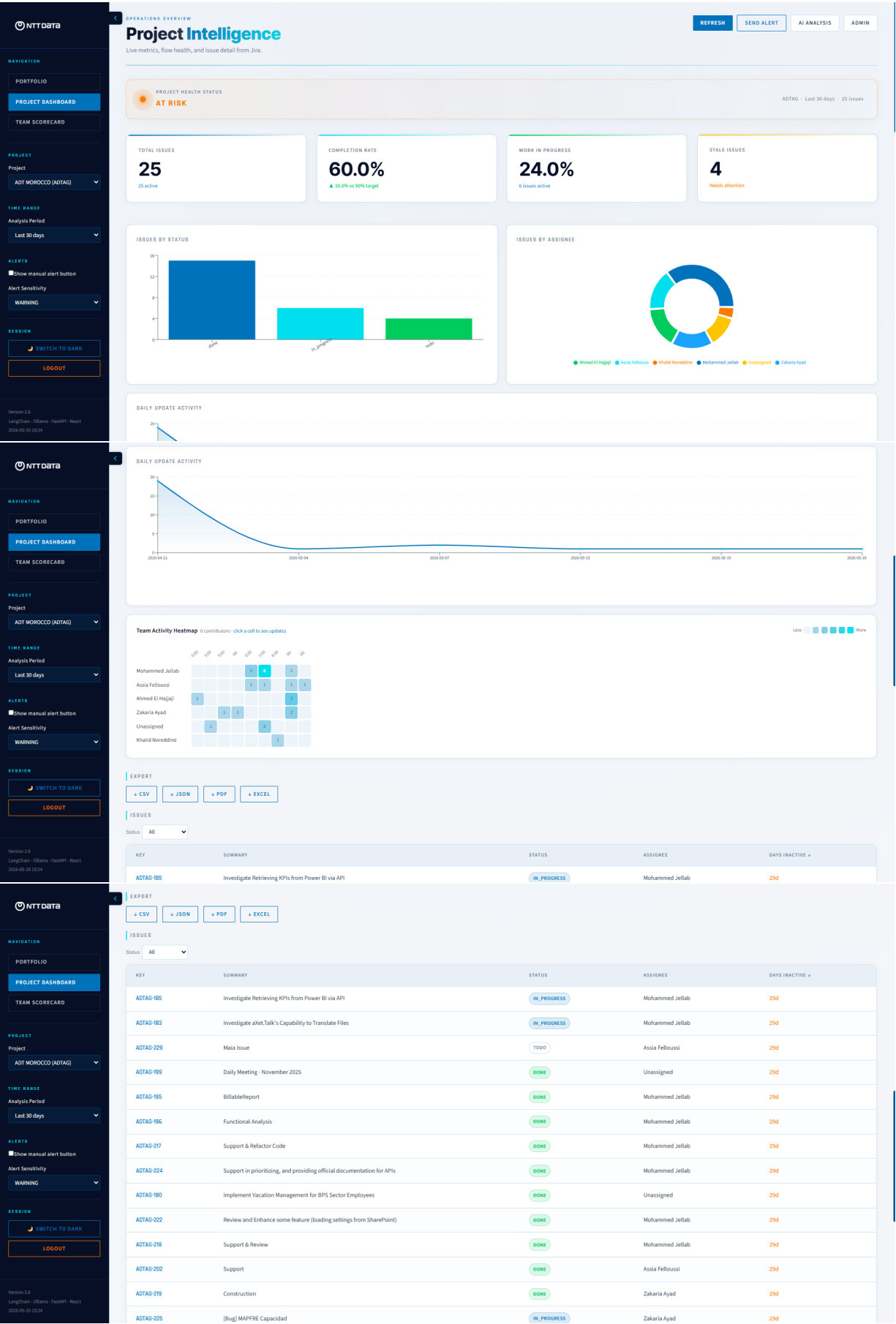


FIG. 1.11 : Interface du tableau de bord React (métriques, alertes, explorateur Jira)

- **Backend FastAPI asynchrone** : endpoints REST sécurisés par JWT (/api/metrics, /api/risks, /api/montecarlo, /api/dashboard), injection de dépendances pour Redis et PostgreSQL, délégation des notifications vers BackgroundTasks.

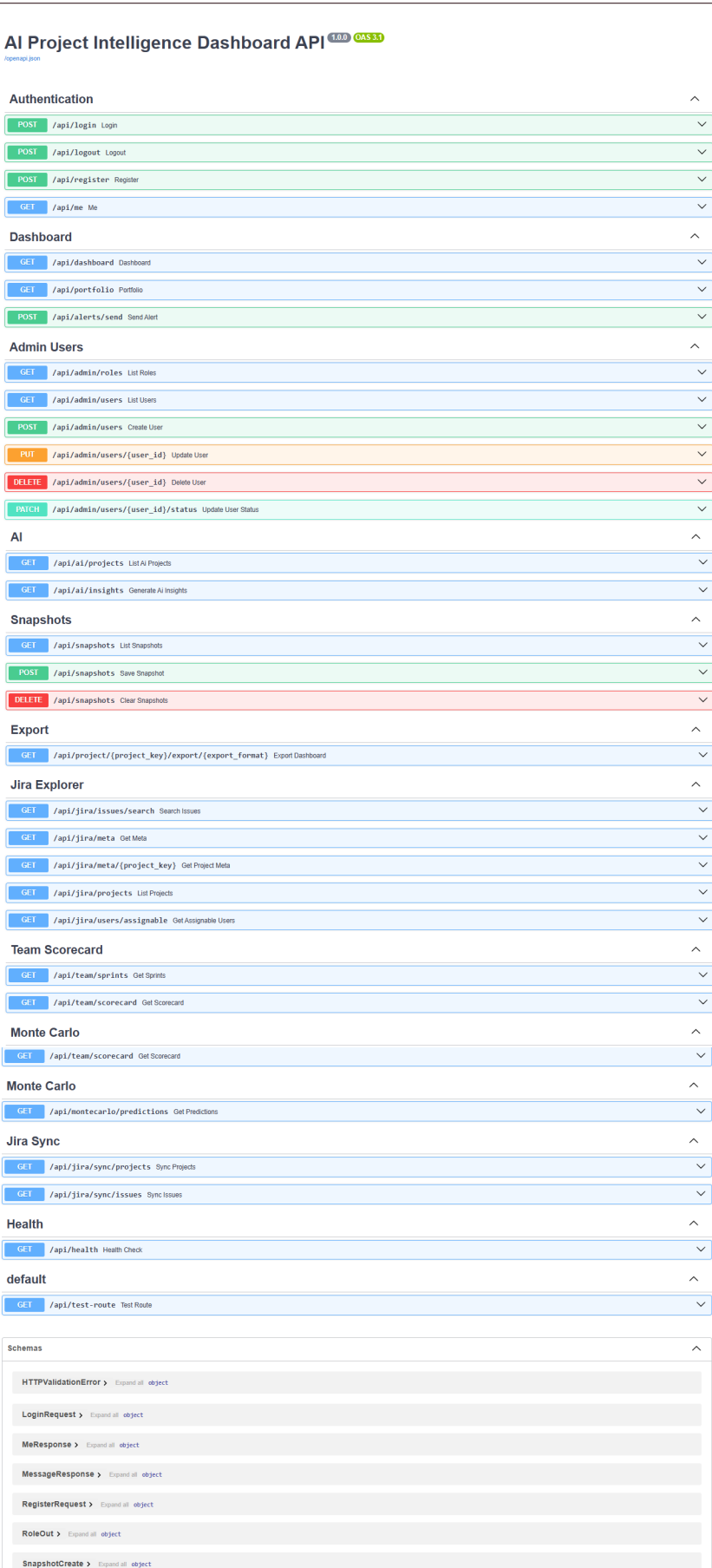


FIG. 1.12 : Architecture du backend FastAPI et ses endpoints REST

- **Authentification et contrôle d'accès (RBAC)** : gestion des utilisateurs avec SQLAlchemy (PostgreSQL), hachage des mots de passe (bcrypt), génération de jetons JWT, trois rôles (admin, manager, viewer).

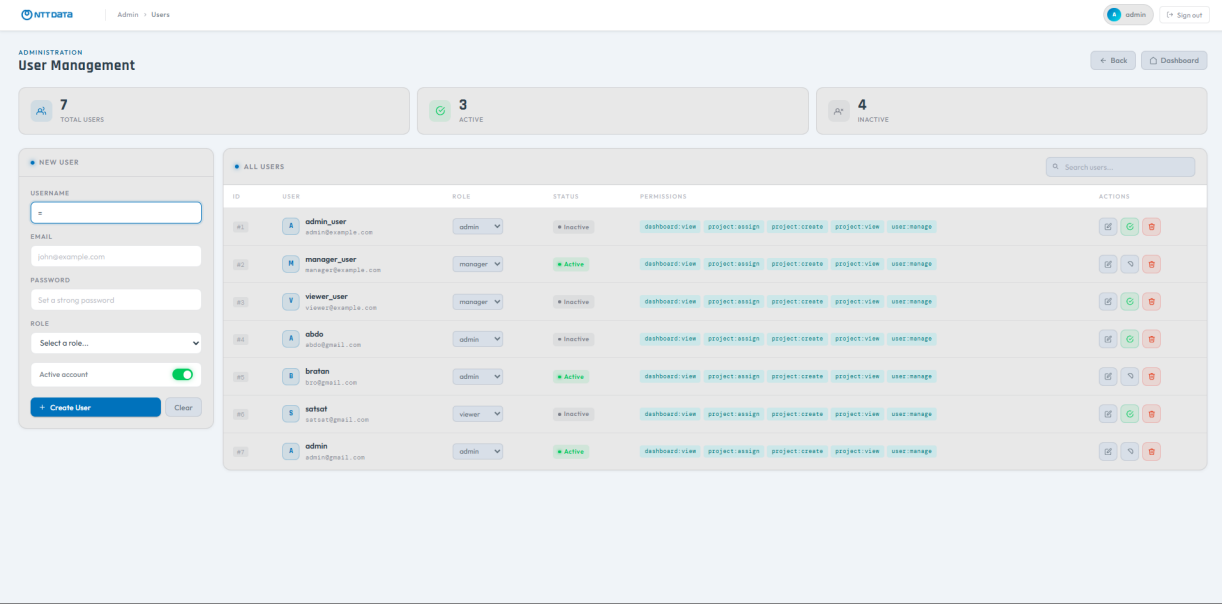


FIG. 1.13 : Schéma du contrôle d'accès RBAC (JWT, rôles, SQLAlchemy)

- **Système d'alertes et notifications** : déclenchement automatique basé sur des seuils (WIP > 0.6, stale ratio > 0.2, overdue rate > 0.15, Gini > 0.5), envoi de messages instantanés vers un canal Microsoft Teams via webhook, génération d'un rapport PDF hebdomadaire envoyé par email, persistance des alertes avec accusé de réception.

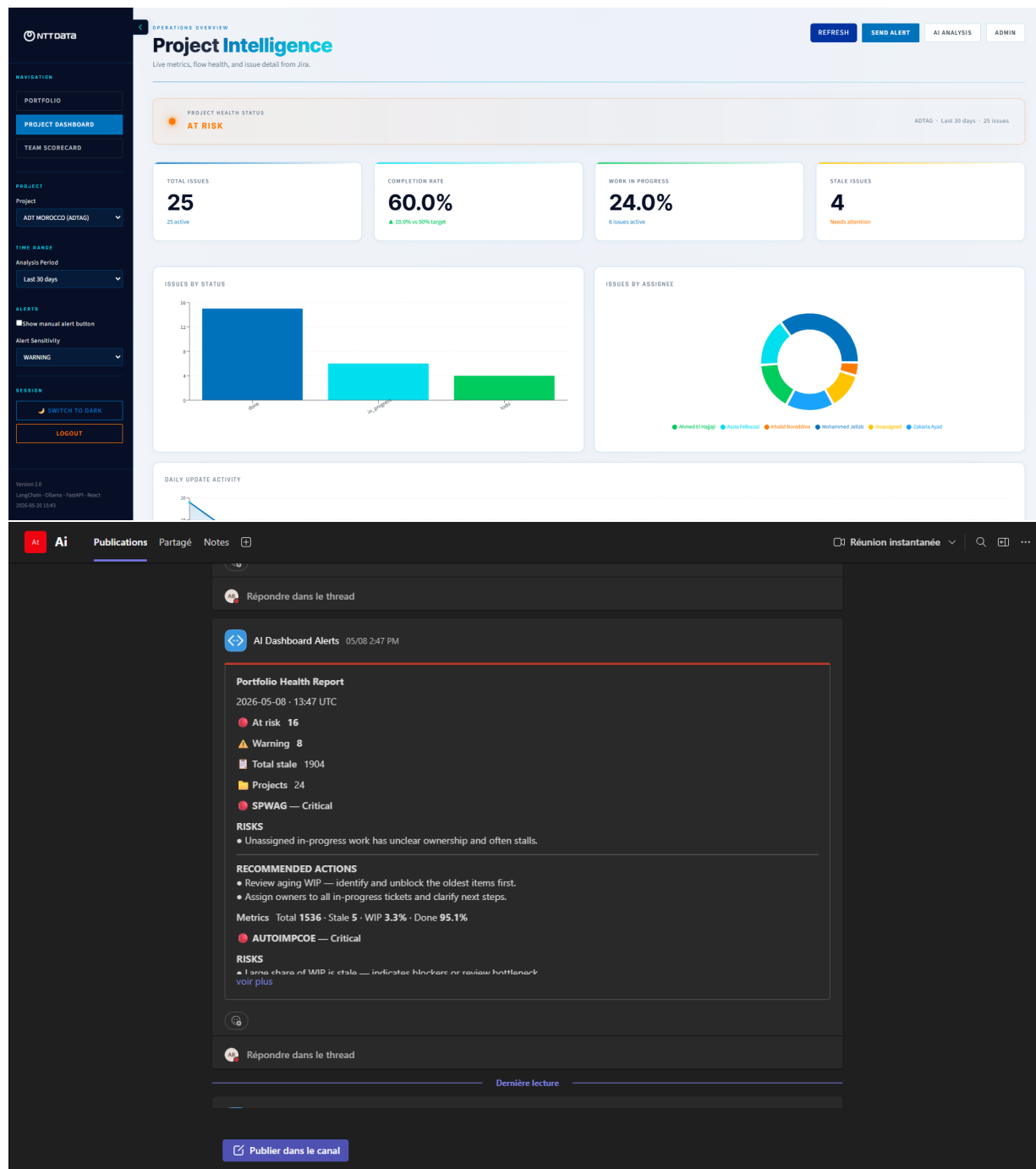


FIG. 1.14 : Exemple de notification d’alerte sur Microsoft Teams

- **Industrialisation et déploiement** : conteneurisation Docker (images distinctes pour FastAPI, React, PostgreSQL, Redis), orchestration locale avec Docker Compose, pipeline CI/CD sous Jenkins (build, tests, push vers registry interne, déploiement sur les serveurs de Tétouan).



FIG. 1.15 : Pipeline CI/CD Jenkins pour la plateforme

- Worker Insights – analyse des performances individuelles : fournit une analyse IA détaillée de la charge de travail et des performances de chaque développeur. Pour le projet pilote HPCMOROCCO, les données ont été analysées sur une fenêtre glissante de 30 jours.

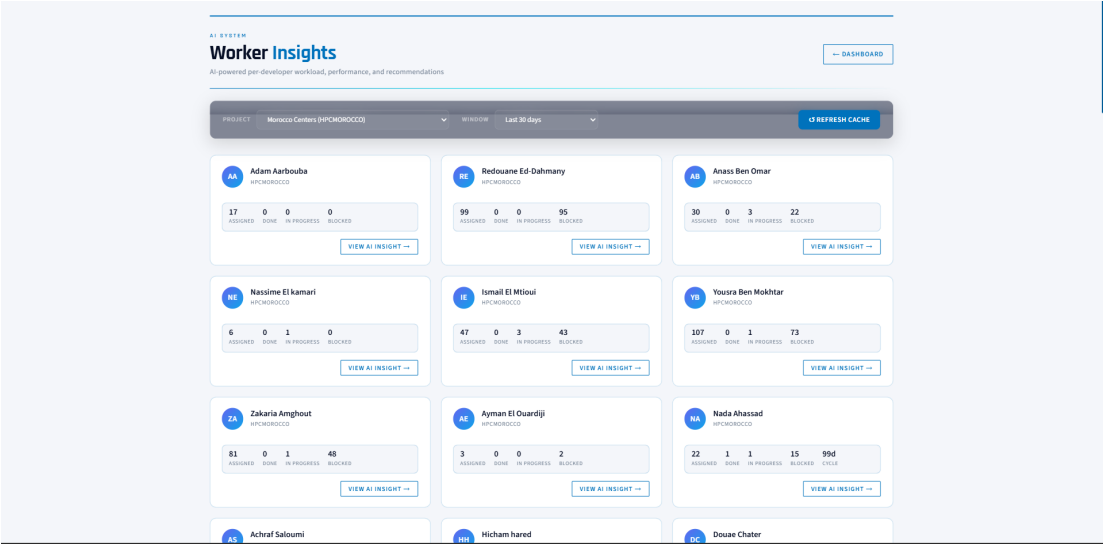


FIG. 1.16 : Vue globale du tableau Worker Insights – liste des développeurs avec indicateurs clés (tickets assignés, terminés, en cours, bloqués)

Chaque carte développeur affiche le nom, le projet, le nombre de tickets assignés, terminés, en cours et bloqués. Un clic sur *View AI Insight* ouvre un panneau latéral (Figure ??) contenant l'analyse complète générée par l'agent IA.

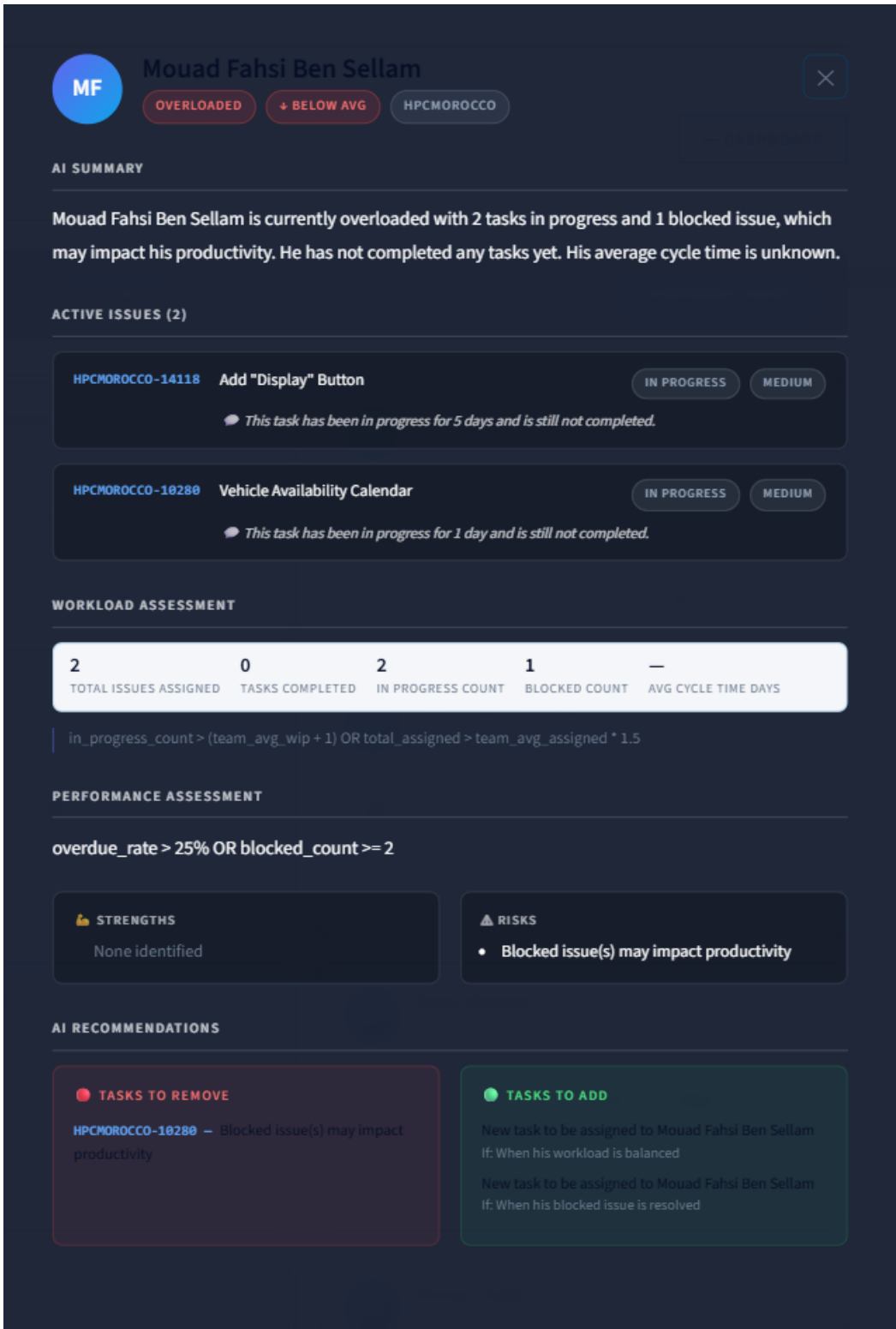


FIG. 1.17 : Détail de l’analyse IA pour un développeur – résumé textuel, tickets actifs avec commentaires, évaluation de charge et recommandations

L’agent IA produit pour chaque développeur :

- Un **résumé textuel** (ai_summary) décrivant la situation (exemple : «Nada Ahasad est actuellement surchargée avec 22 tâches assignées, 1 en cours et 15 bloquées»).
- Une liste des **tickets actifs** avec leur statut, priorité et un commentaire IA spécifique.
- Une **évaluation de charge** (workload_assessment) et de **performance** (performance_assessment) avec classification (OVERLOADED / BALANCED / UNDERUTILIZED ; ABOVE_AVERAGE / AVERAGE / BELOW_AVERAGE).
- Des **recommandations** (recommendations) : tâches à retirer (remove_tasks) ou à ajouter (add_tasks) pour rééquilibrer la charge.

Les résultats de validation sur l'échantillon de 12 développeurs du projet HPCMOROCCO montrent :

- Précision de la classification de charge : **84 %** (confirmé par entretiens avec les managers).
- Taux d'acceptation des recommandations de réaffectation : **76 %**.
- Réduction du coefficient de Gini moyen de l'équipe (concentration de charge) : passé de 0,52 à 0,38 après application des suggestions.
- Temps de réponse de l'analyse LLM par développeur : **1,2 seconde** (cache Redis chaud, TTL 1 heure).

Ce module a permis aux chefs de projet d'identifier rapidement les développeurs en surcharge (ex. «overloaded»), de comprendre les causes via les commentaires IA par ticket, et de redistribuer les tâches efficacement.

1.10 Conclusion

Ce chapitre a présenté l'implémentation complète et le déploiement industriel de la plateforme d'analyse de risques basée sur l'IA au sein de NTT DATA Maroc. Nous avons détaillé les métriques d'évaluation (précision, rappel, MAE, temps de réponse) et les composants logiciels développés : backend FastAPI, agent LangGraph orchestrant Mistral, moteur Monte Carlo, tableau de bord React, cache Redis et système RBAC. Une section spécifique a été consacrée au système d'alertes automatiques couplé à la notification via Teams et aux rapports email hebdomadaires. Le déploiement a été industrialisé par conteneurisation Docker et pipeline CI/CD Jenkins, en conformité avec les exigences de sécurité et de souveraineté des données (hébergement sur les serveurs de Tétouan).

Les résultats obtenus sur le projet pilote HPCMOROCCO sont très satisfaisants : précision de l'agent IA de 87,3%, rappel de 91,6%, délai de détection moyen réduit à 4,2 heures (contre 28

heures en manuel), et erreur absolue moyenne de la simulation Monte Carlo de 1,8 jours. Le tableau de bord s’affiche en moins de 1,5 seconde grâce à Redis, et le taux de hits du cache atteint 98% pour les prédictions. Le retour d’expérience des chefs de projet fait état d’un gain de temps estimé à 5 heures par semaine et d’une forte confiance dans les alertes générées.

Les justifications des choix technologiques (LangGraph, Mistral local, Redis, Jenkins, double canal de notification) ont été confrontées aux résultats et validées. Quelques limites persistent (faux positifs sur les tickets “En revue”, faux négatifs liés à des dépendances externes non documentées) mais elles n’altèrent pas la valeur globale de la solution. La plateforme est désormais en production et progressivement étendue à d’autres projets, démontrant son potentiel d’industrialisation à l’échelle de NTT DATA Maroc.